

HỆ THỐNG MÁY TÍNH

Phạm Tuấn Sơn – ptson@fit.hcmus.edu.vn

08 – Thiết kế CPU

Quan điểm về cấu tạo CPU

- William Stallings

- Registers
- ALU
- CU
- Internal bus

Mục tiêu: hiểu được cấu tạo và hoạt động của CPU

- Patterson & Hennessy

- Datapath
- Control

Mục tiêu: thiết kế CPU

Các bước thiết kế một CPU

1. Phân tích kiến trúc tập lệnh (ISA) $\Rightarrow$ yêu cầu về datapath
 - Trình bày từng lệnh dưới dạng register transfers language (RTL) để thấy rõ ý nghĩa các các lệnh
 - Datapath phải có thành phần lưu trữ (bộ nhớ chính / cache) cho các thanh ghi trong kiến trúc bộ lệnh
 - Datapath phải hỗ trợ thực thi tất cả các lệnh
2. Lựa chọn và lắp ráp các khối mạch (tổ hợp, tuần tự) cần thiết để xây dựng datapath đáp ứng yêu cầu tập lệnh.
 - Chia nhỏ quá trình xử lý một lệnh thành các công đoạn (stage) nhỏ (nạp lệnh $\rightarrow$ giải mã lệnh $\rightarrow$ tính toán $\rightarrow$...), xây dựng khối xử lý cho từng công đoạn rồi lắp ráp thành datapath.
 - Các công đoạn nhỏ dễ thiết kế.
 - Dễ thay đổi, tối ưu một công đoạn mà ít ảnh hưởng tới các công đoạn khác.
3. Phân tích mỗi lệnh để xác định các tín hiệu điều khiển cần thiết; từ đó, thiết kế mạch control cho các tín hiệu điều khiển.

Thiết kế mạch xử lý LEGv8 thu gọn

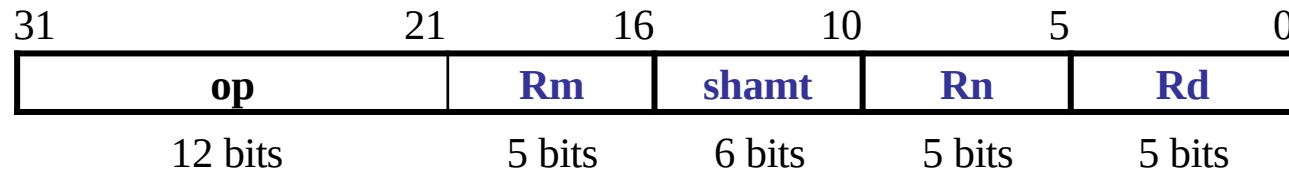
- Mạch xử lý LEGv8 thu gọn gồm 8 lệnh:

ADD Rd, Rn, Rm

SUB Rd, Rn, Rm

AND Rd, Rn, Rm

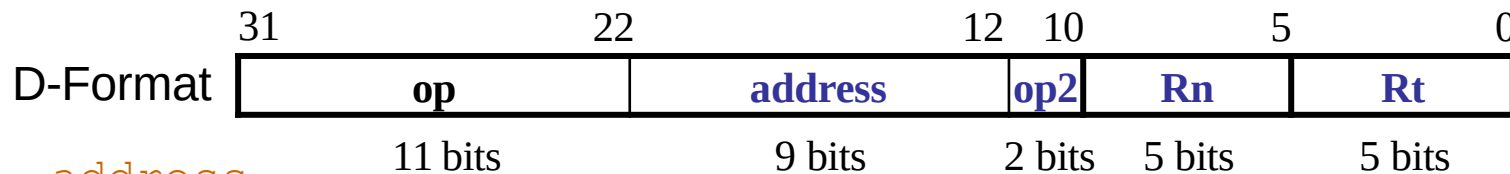
ORR Rd, Rn, Rm



R-Format

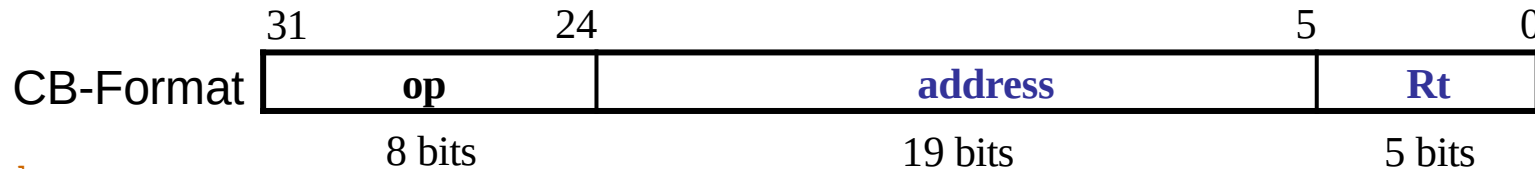
LDUR Rt, [Rn, #address]

STUR Rt, [Rn, #address]



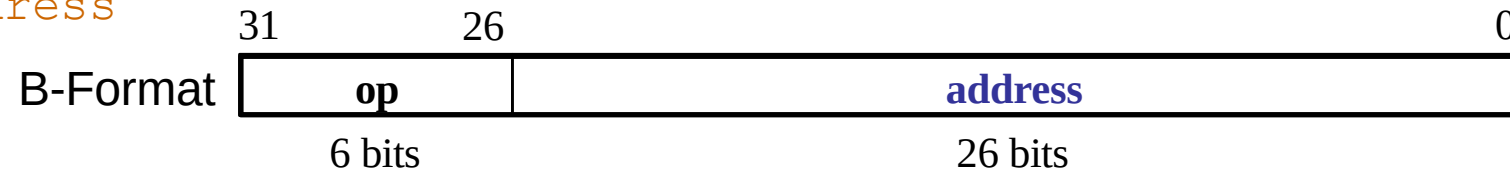
D-Format

CBZ Rt, address



CB-Format

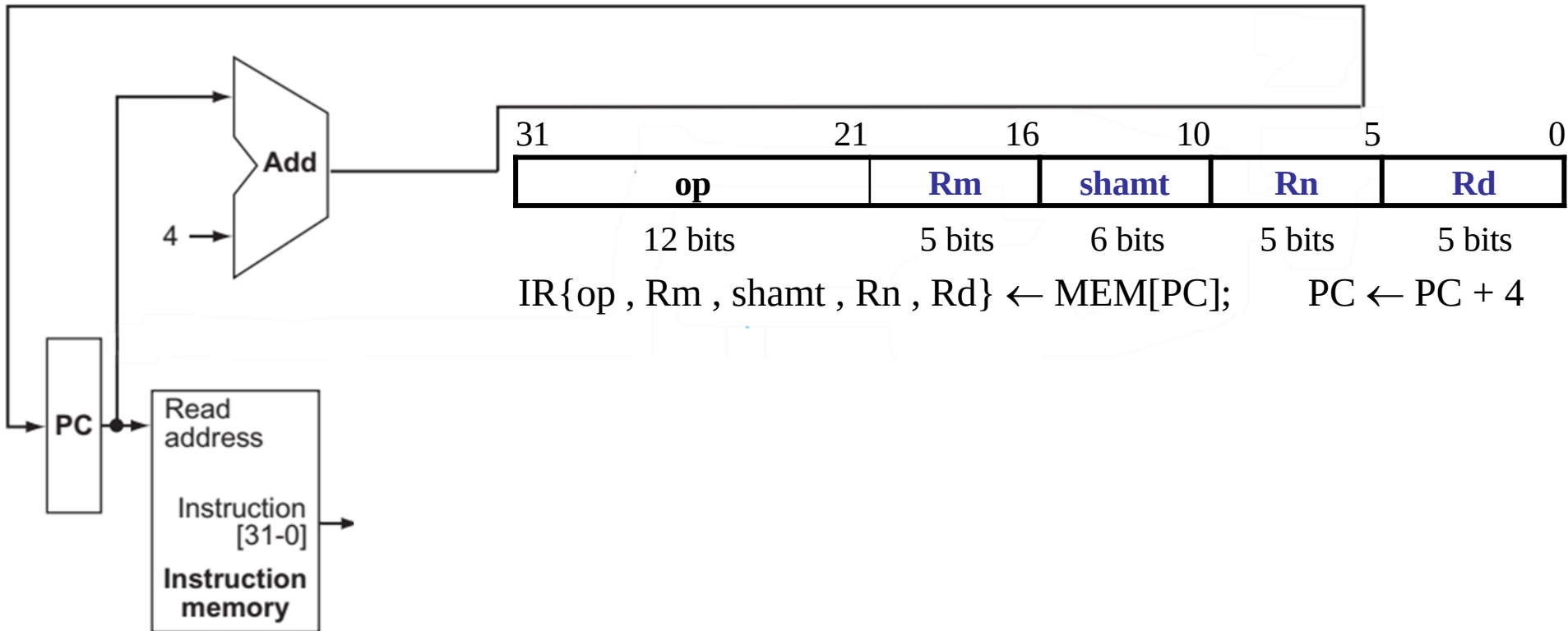
B address



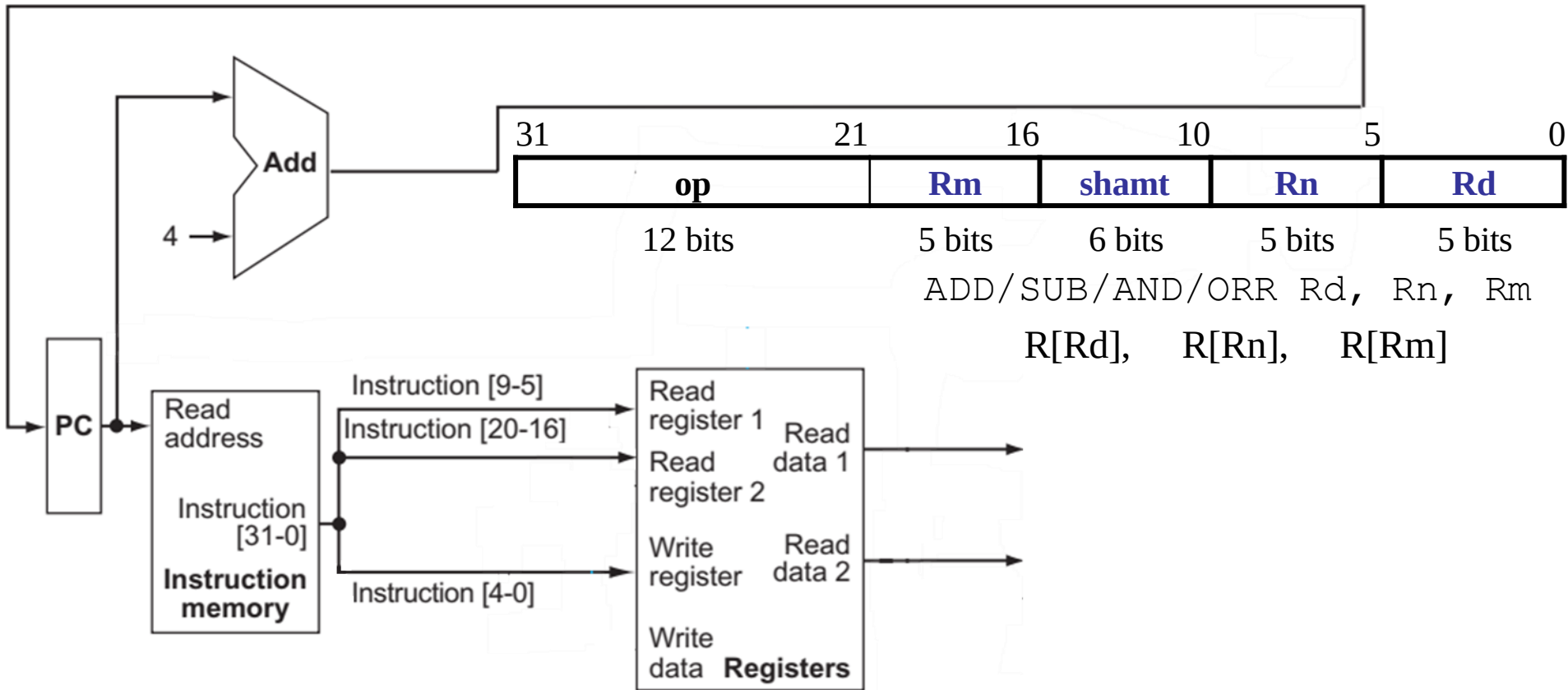
B-Format

- Tại sao là 8 lệnh này ?

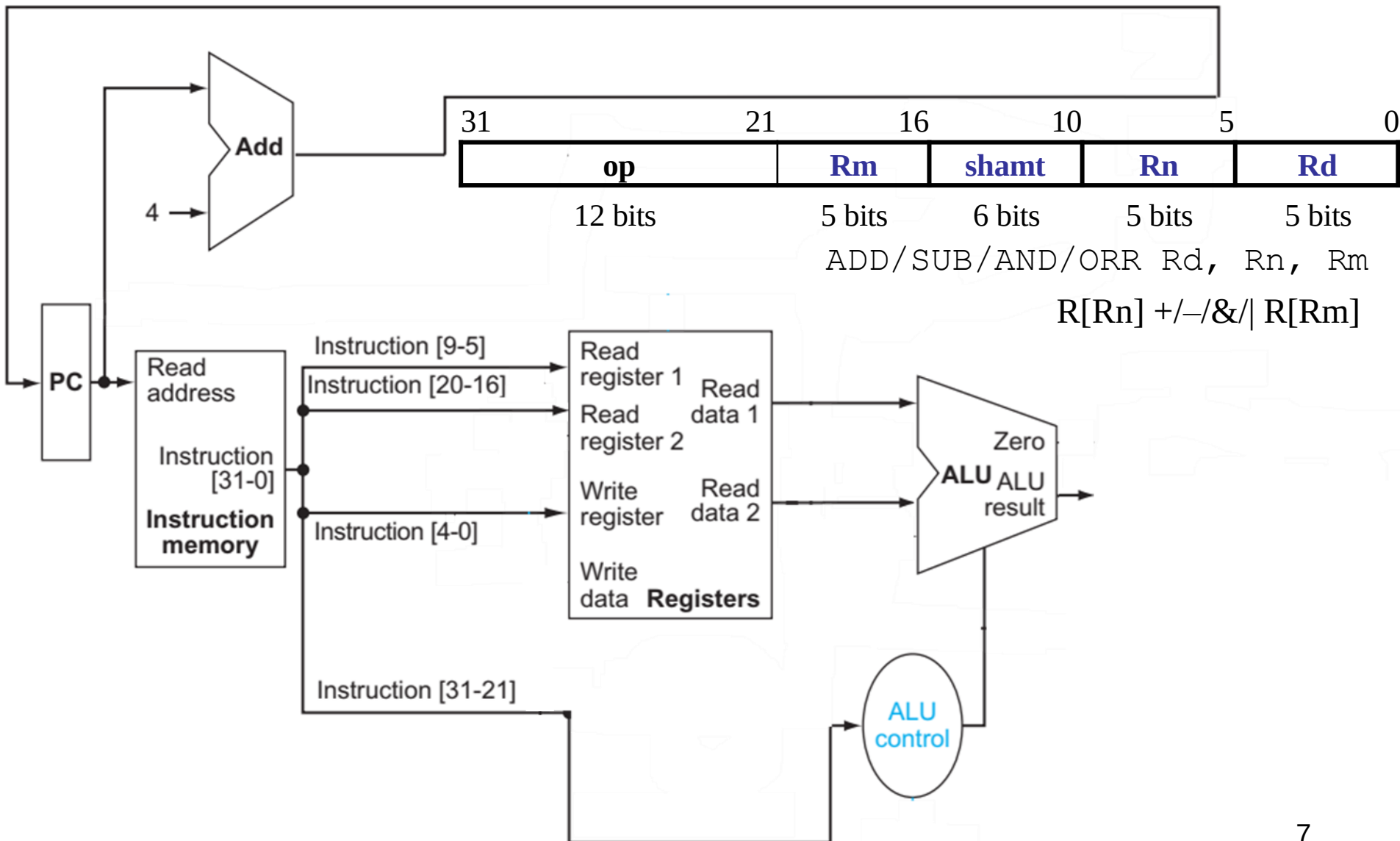
Datapath cho các lệnh R-Format Công đoạn nạp lệnh



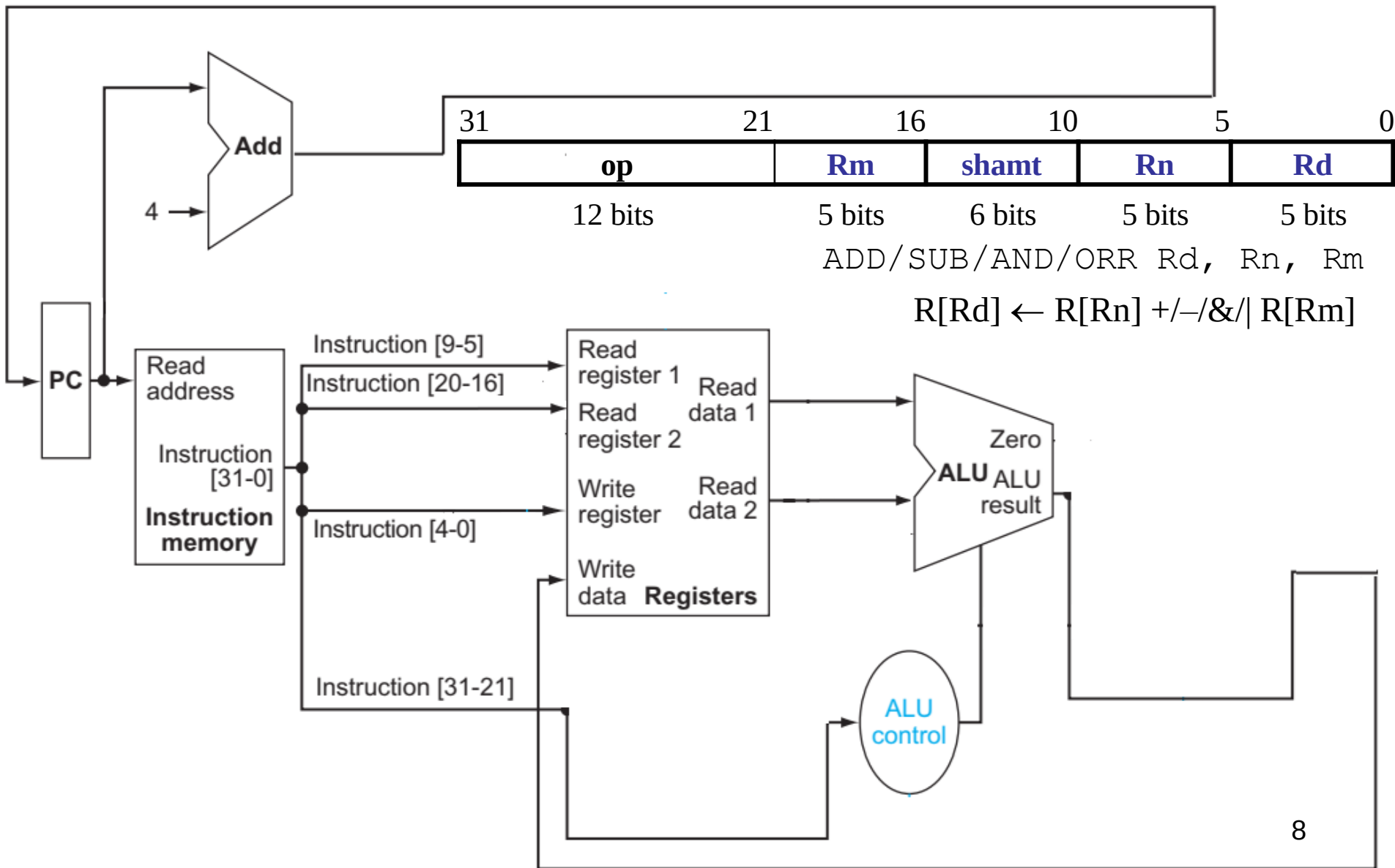
Datapath cho các lệnh R-Format Công đoạn giải mã lệnh



Datapath cho các lệnh R-Format Công đoạn tính toán

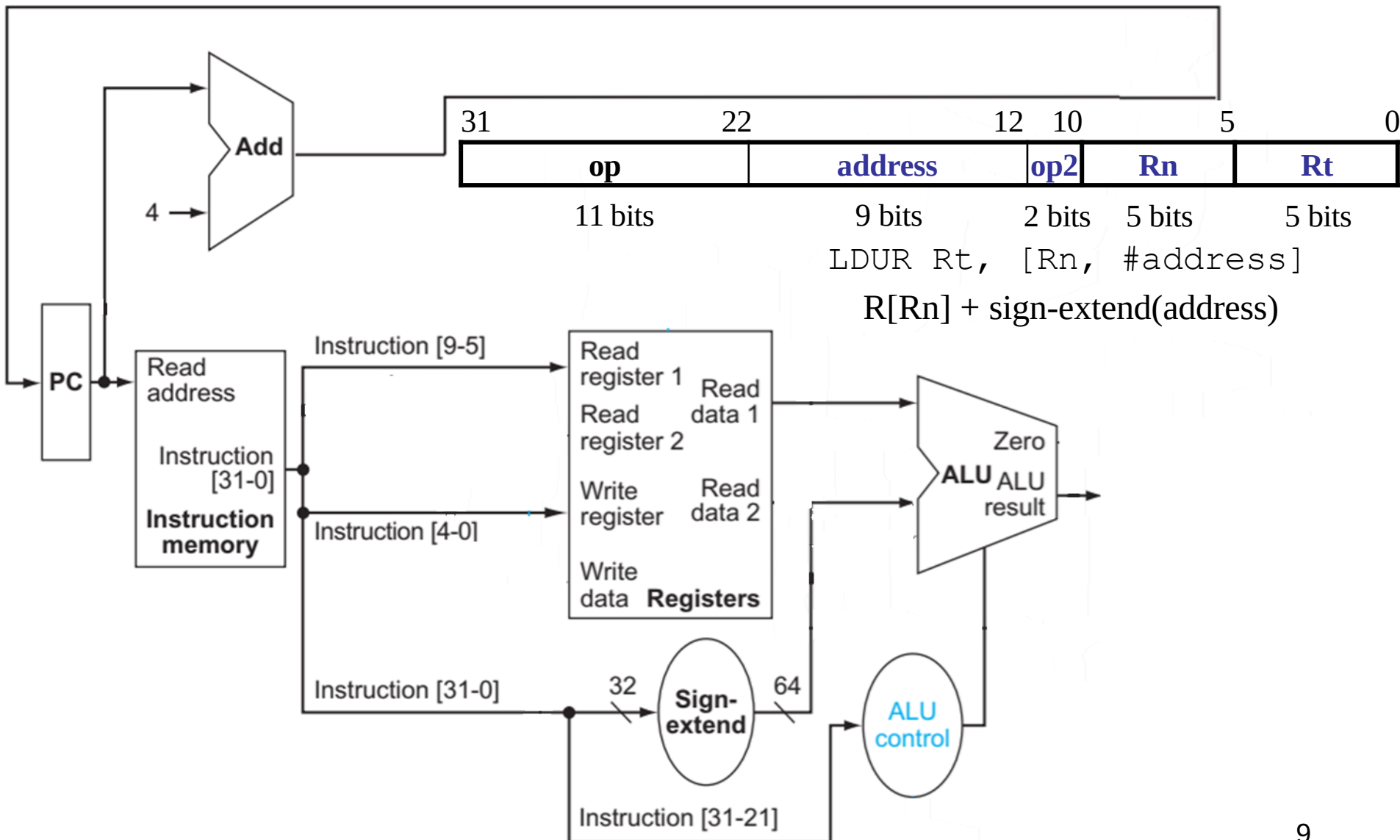


Datapath cho các lệnh R-Format Công đoạn lưu kết quả vào thanh ghi



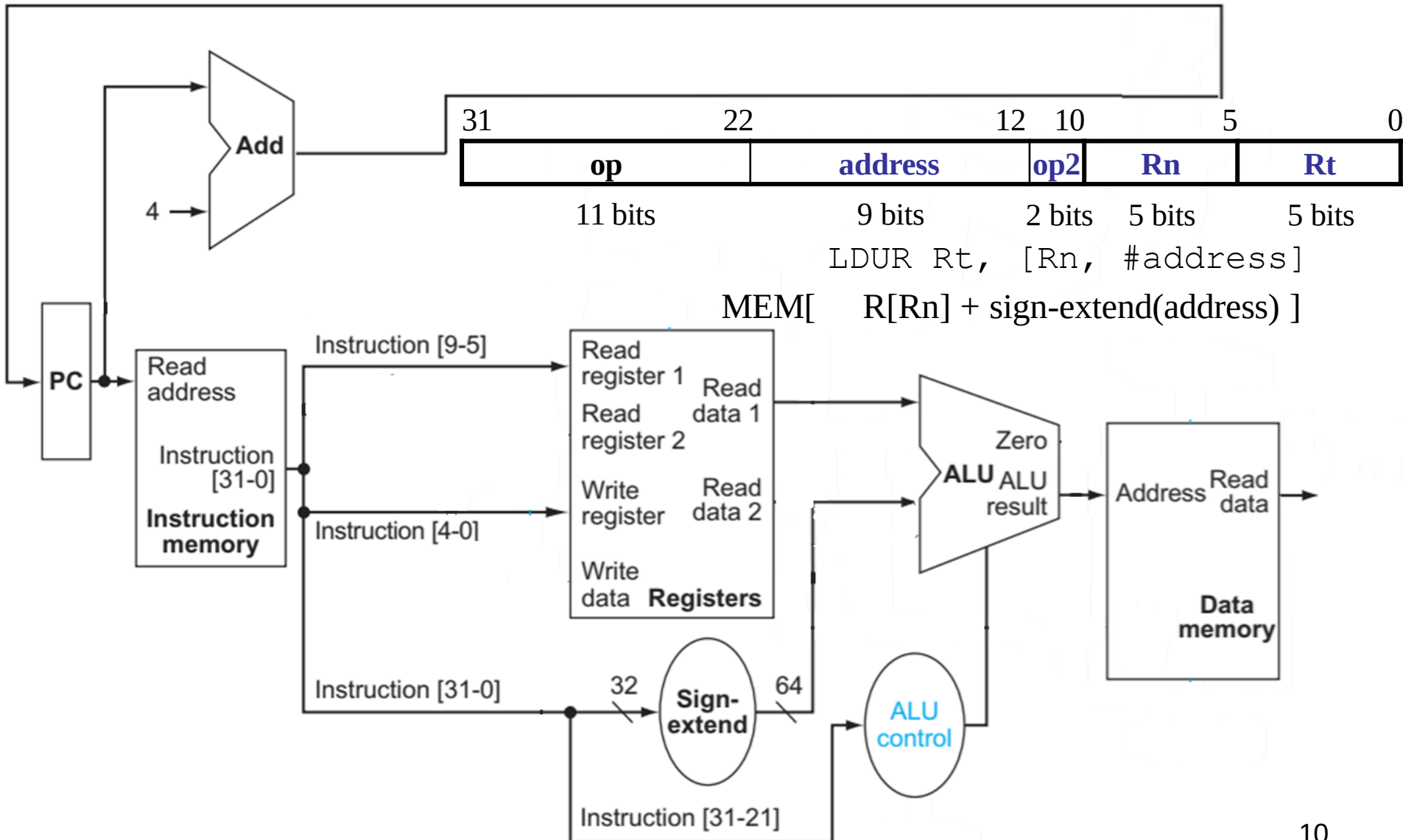
Datapath cho các lệnh LDUR

Công đoạn nạp lệnh, giải mã và tính toán



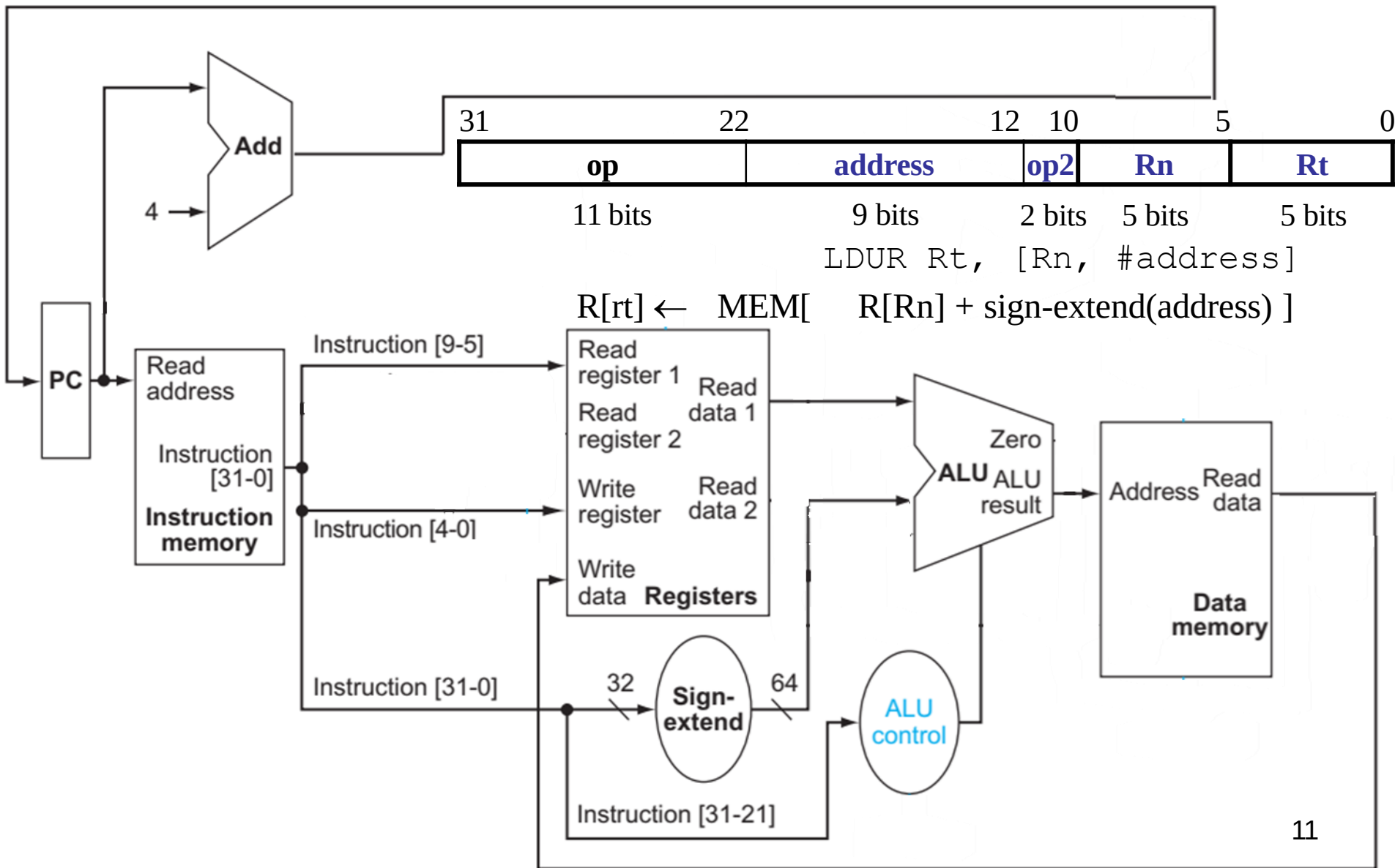
Datapath cho các LDUR

Công đoạn truy xuất bộ nhớ



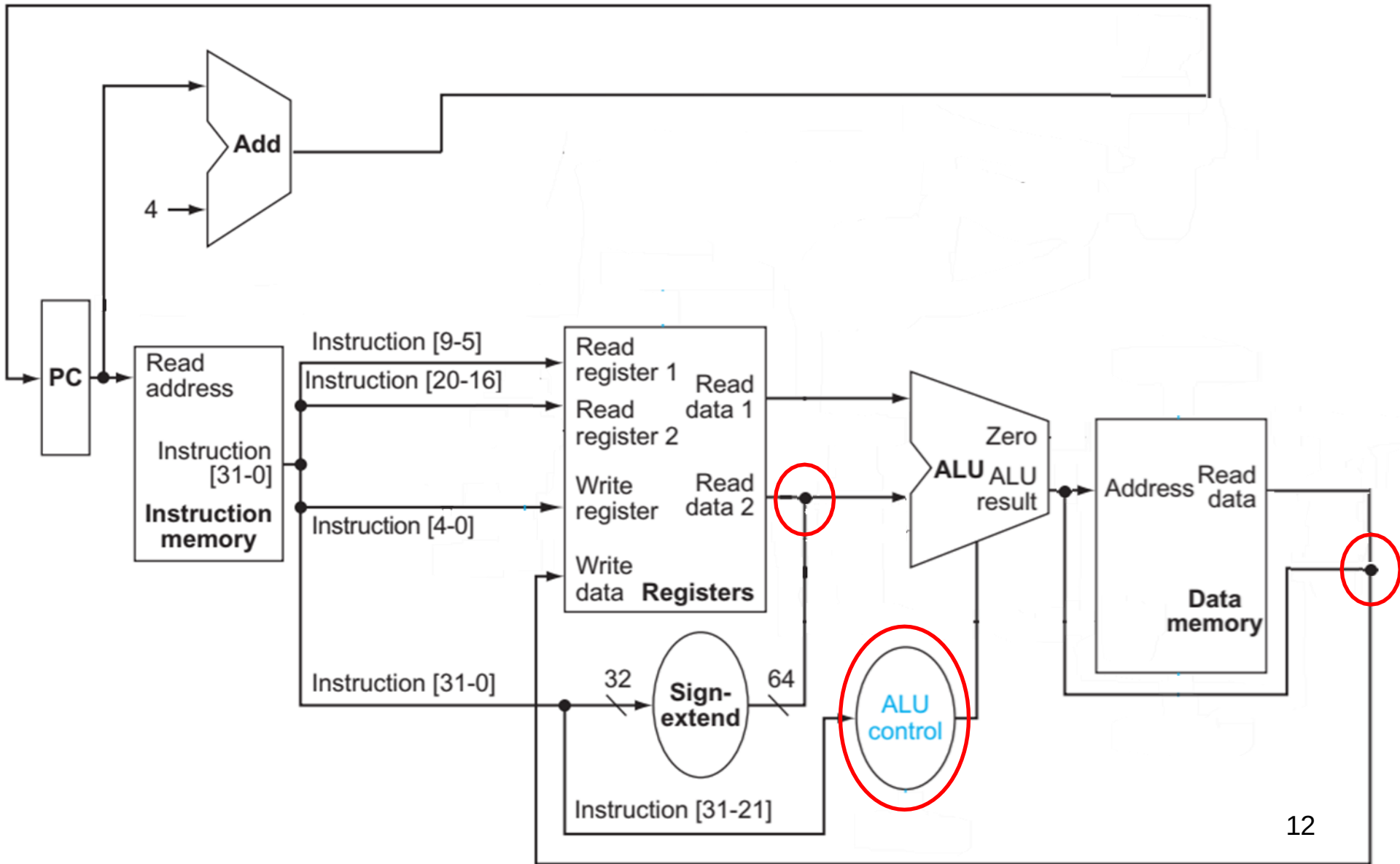
Datapath cho các LDUR

Công đoạn lưu kết quả vào thanh ghi

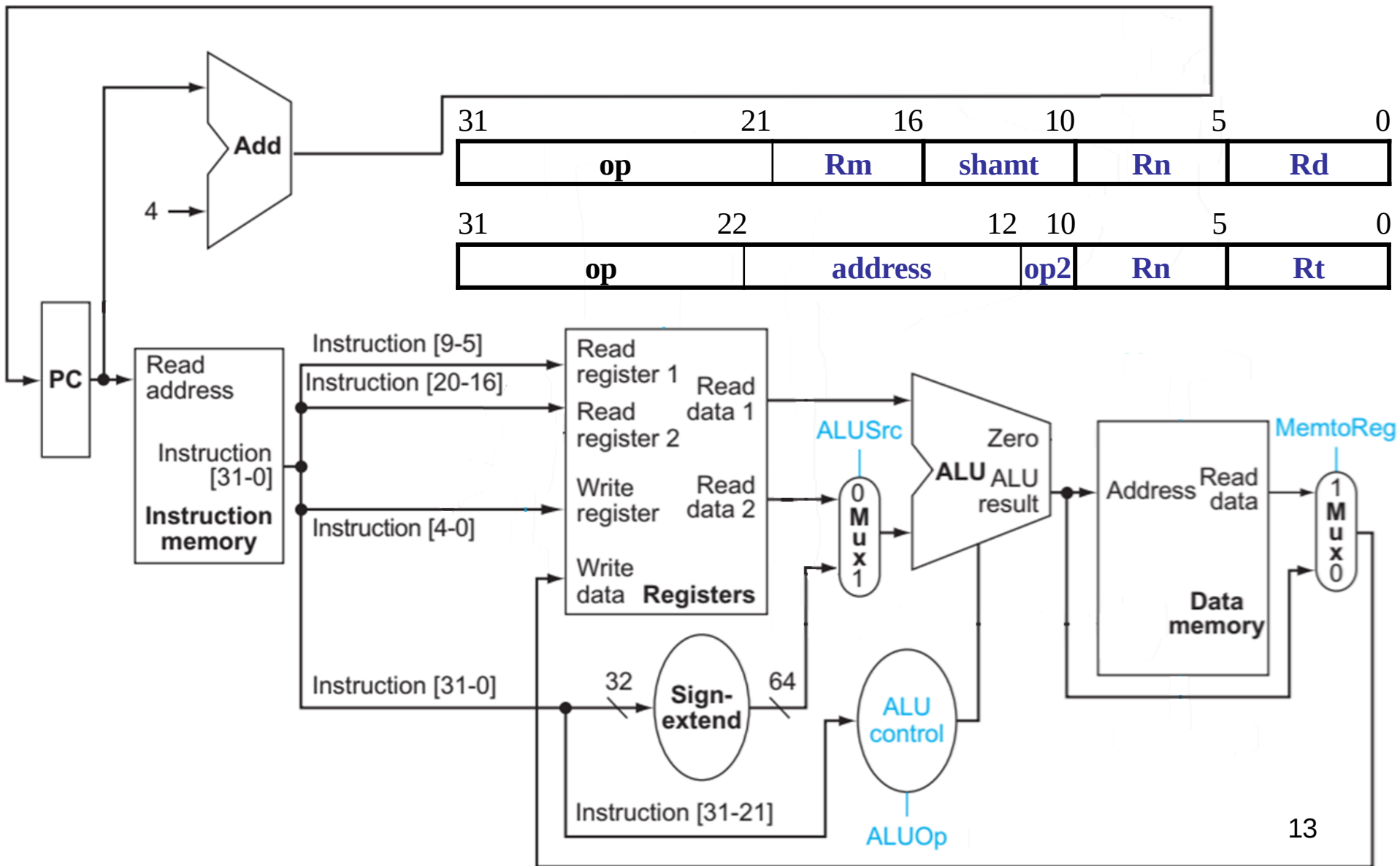


Datapath

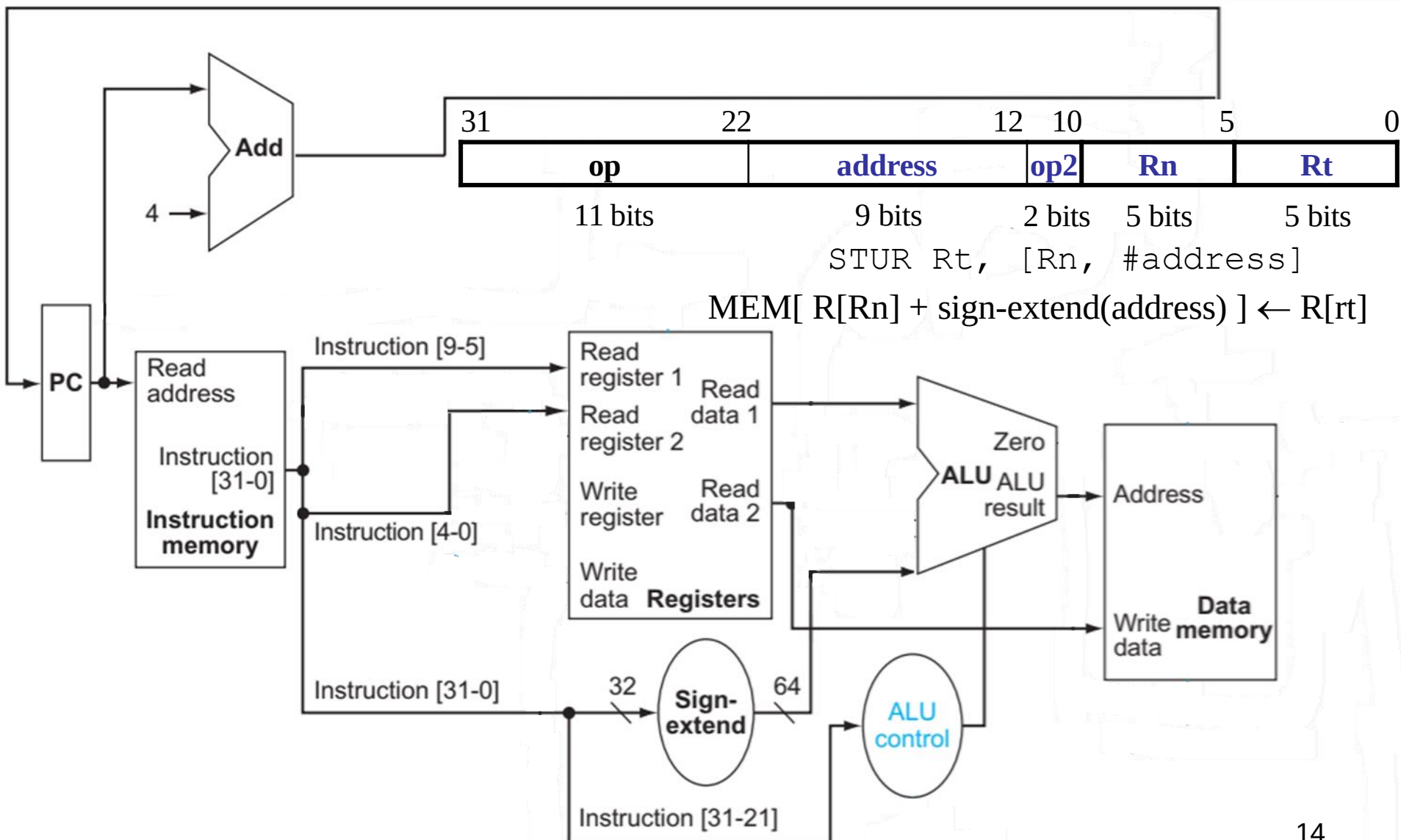
Kết hợp các lệnh R-Format và lệnh LDUR



Datapath cho các lệnh R-Format và lệnh LDUR

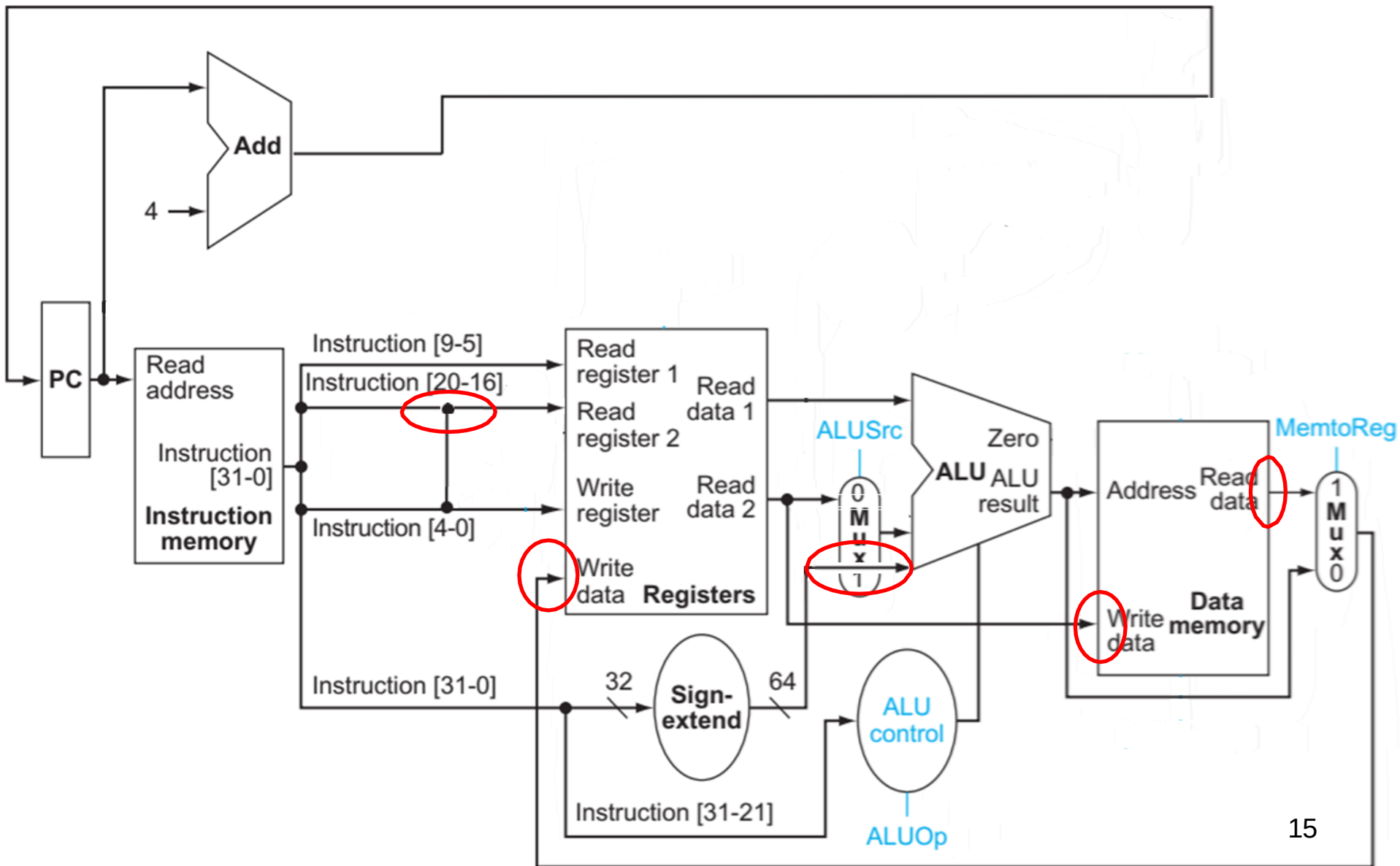


Datapath cho lệnh STUR

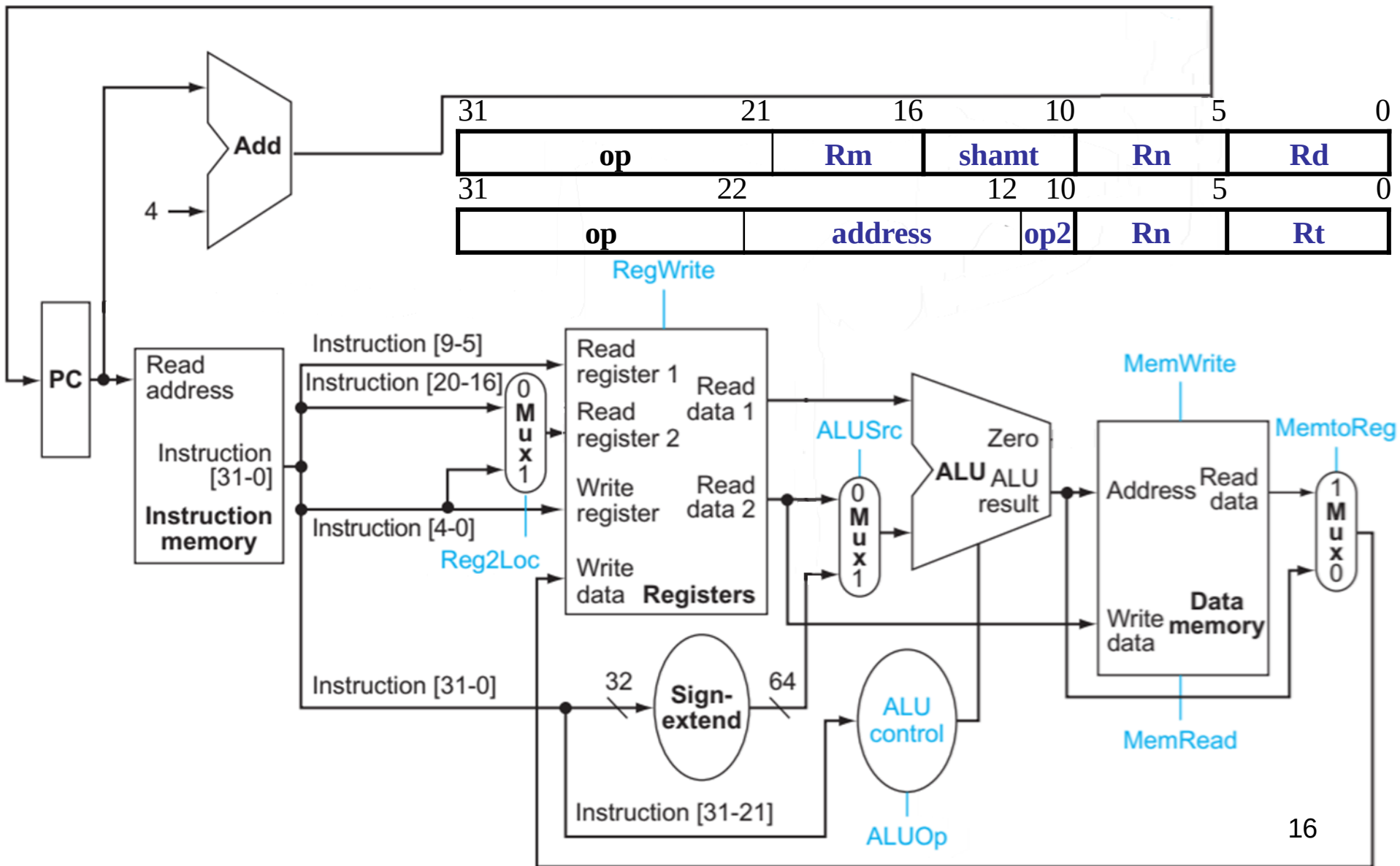


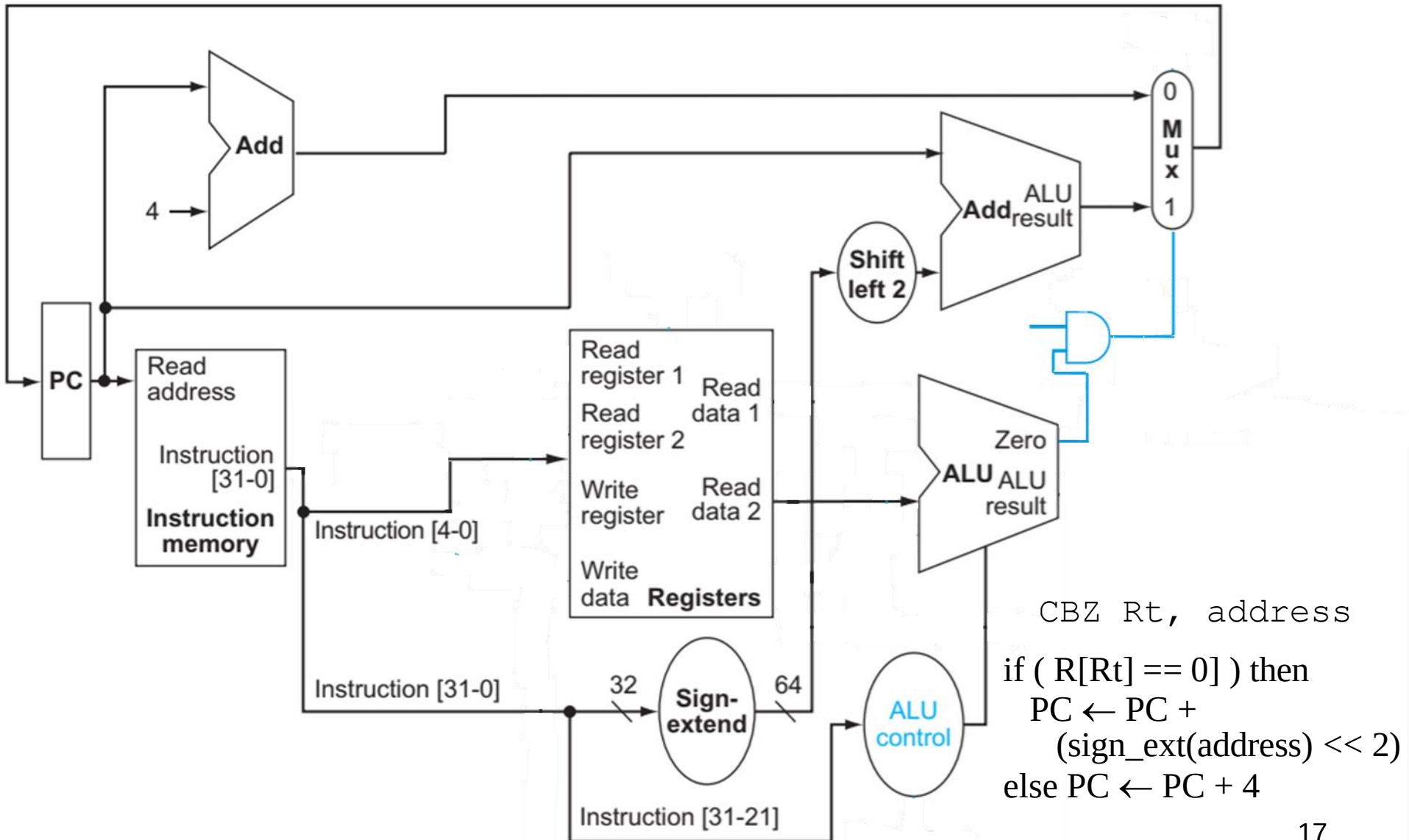
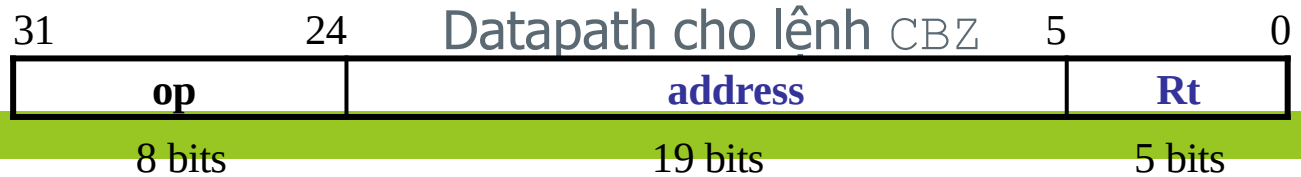
Datapath

Kết hợp lệnh R-Format, LDUR và STUR



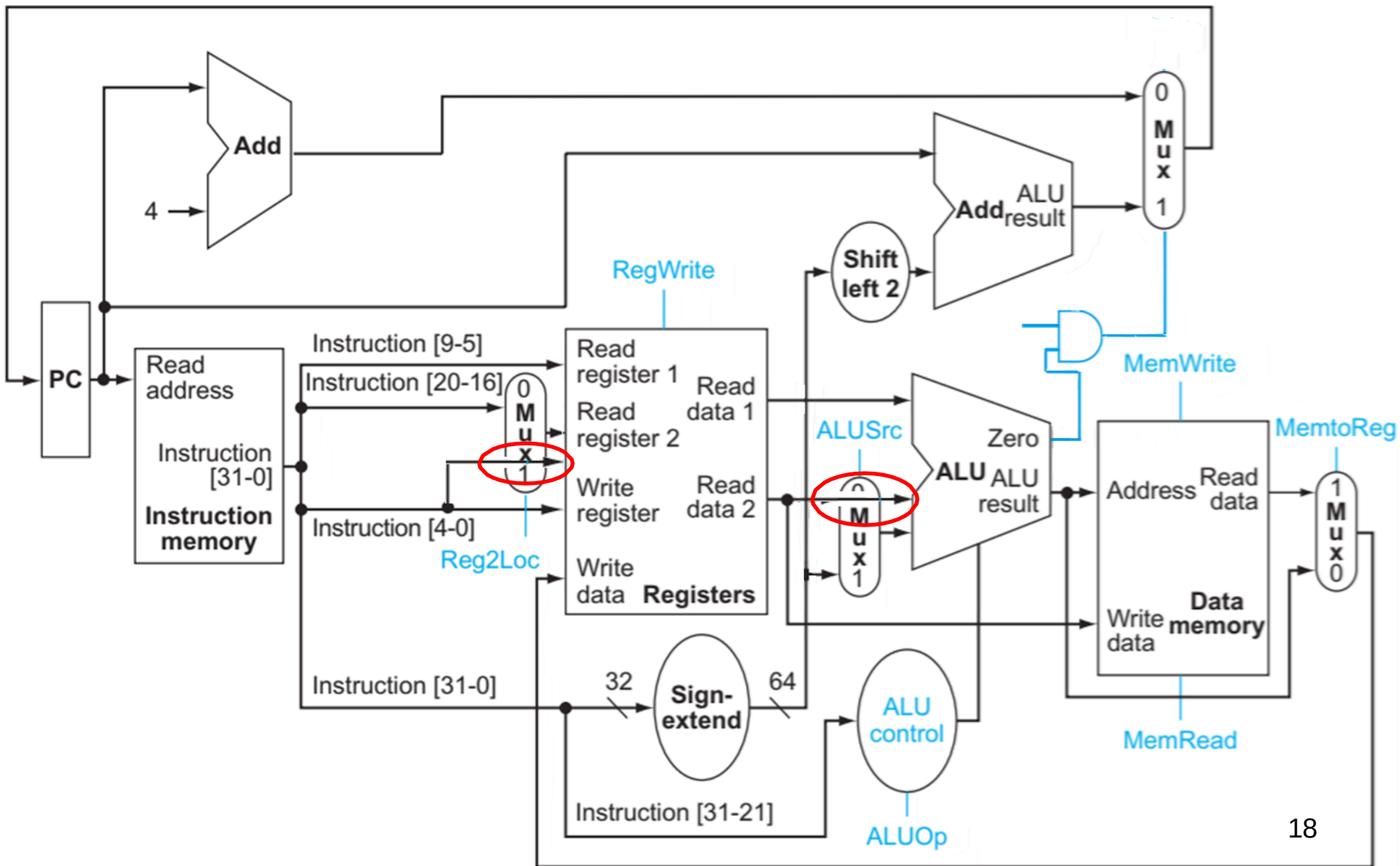
Datapath cho các lệnh R-Format, lệnh LDUR và STUR



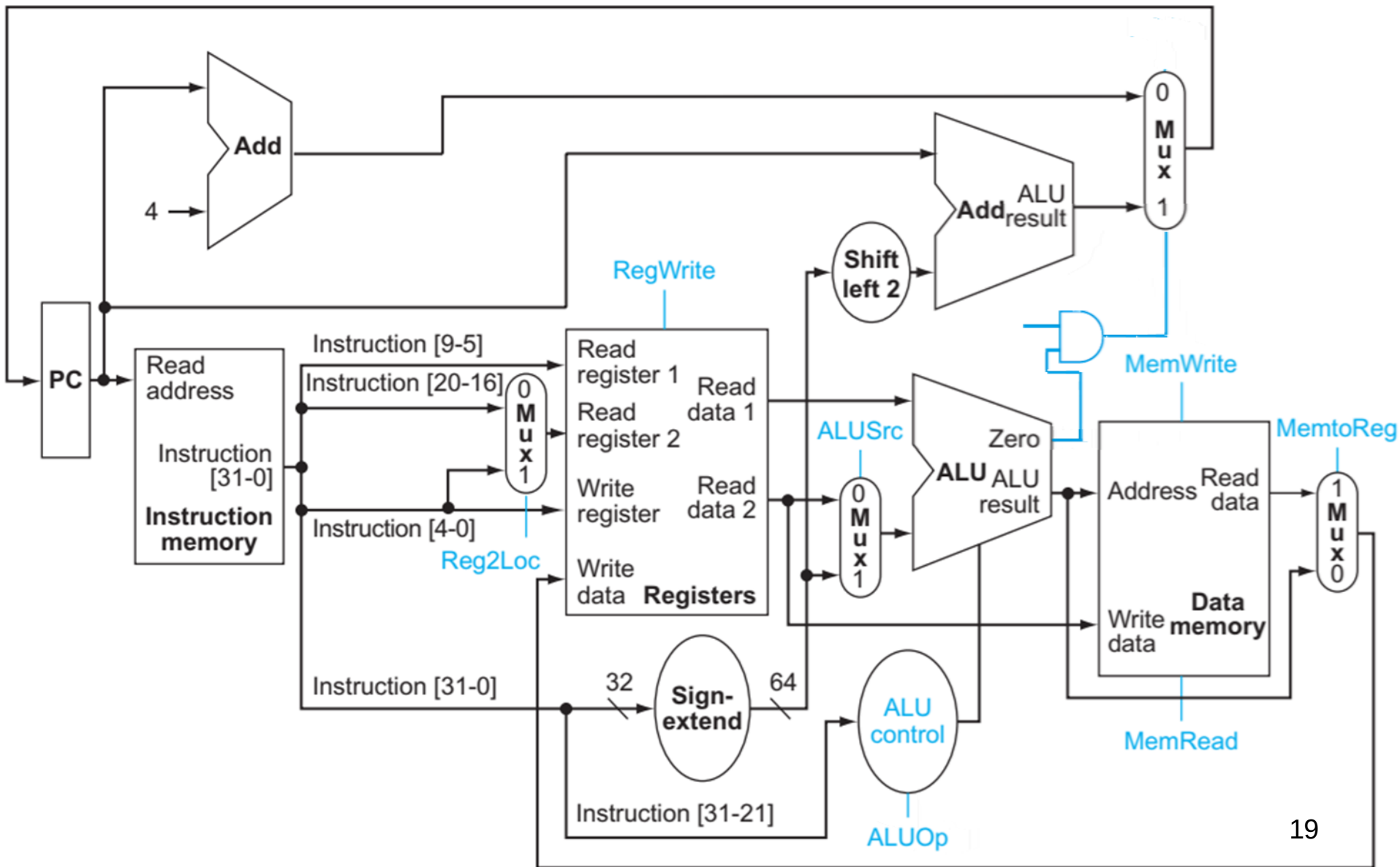


Datapath

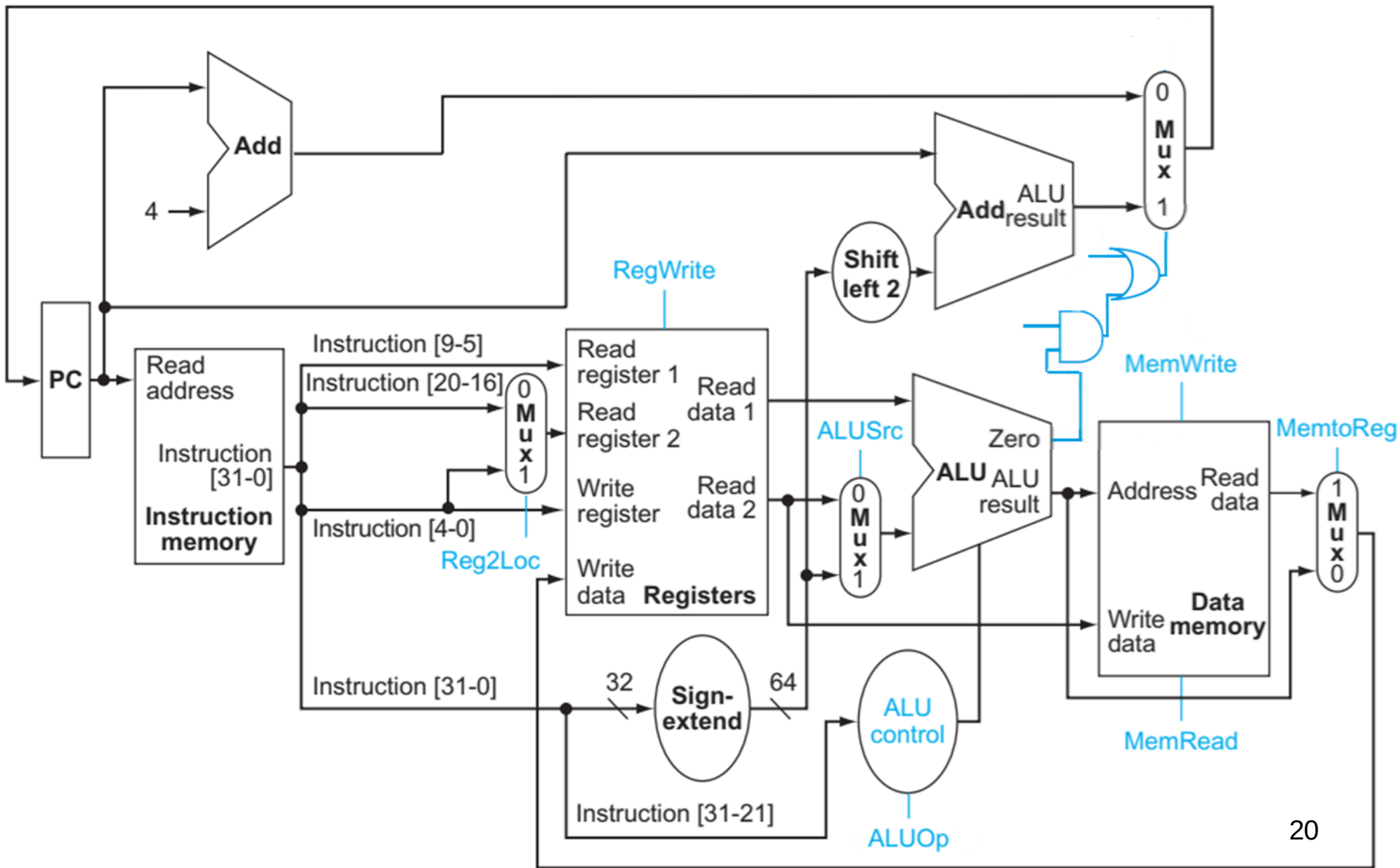
Kết hợp lệnh R-Format, LDUR, STUR và CBZ



Datapath cho các lệnh R-Format, lệnh LDUR, STUR và CBZ



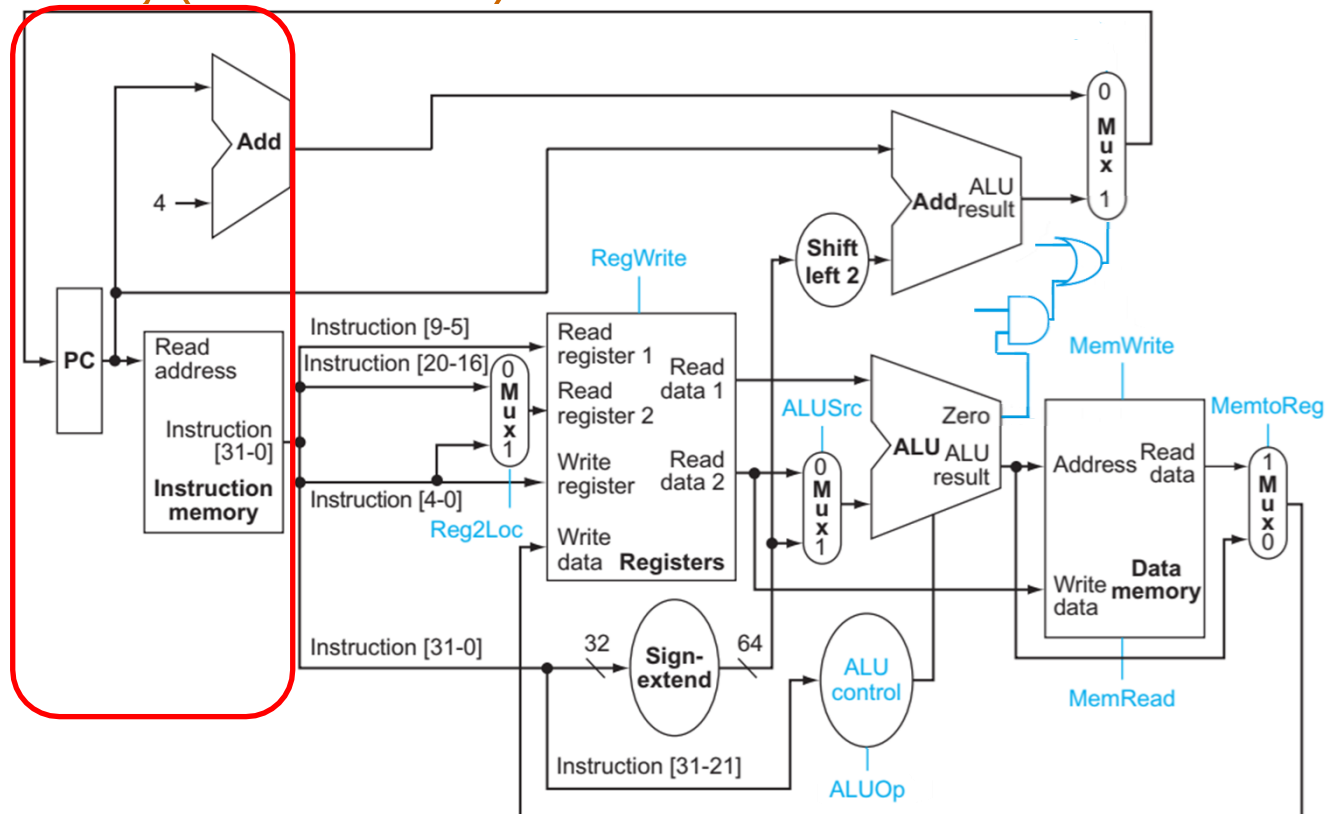
Datapath cho tất cả lệnh LEGv8 thu gọn R-Format, LDUR, STUR, CBZ và B



Các công đoạn lệnh (1/5)

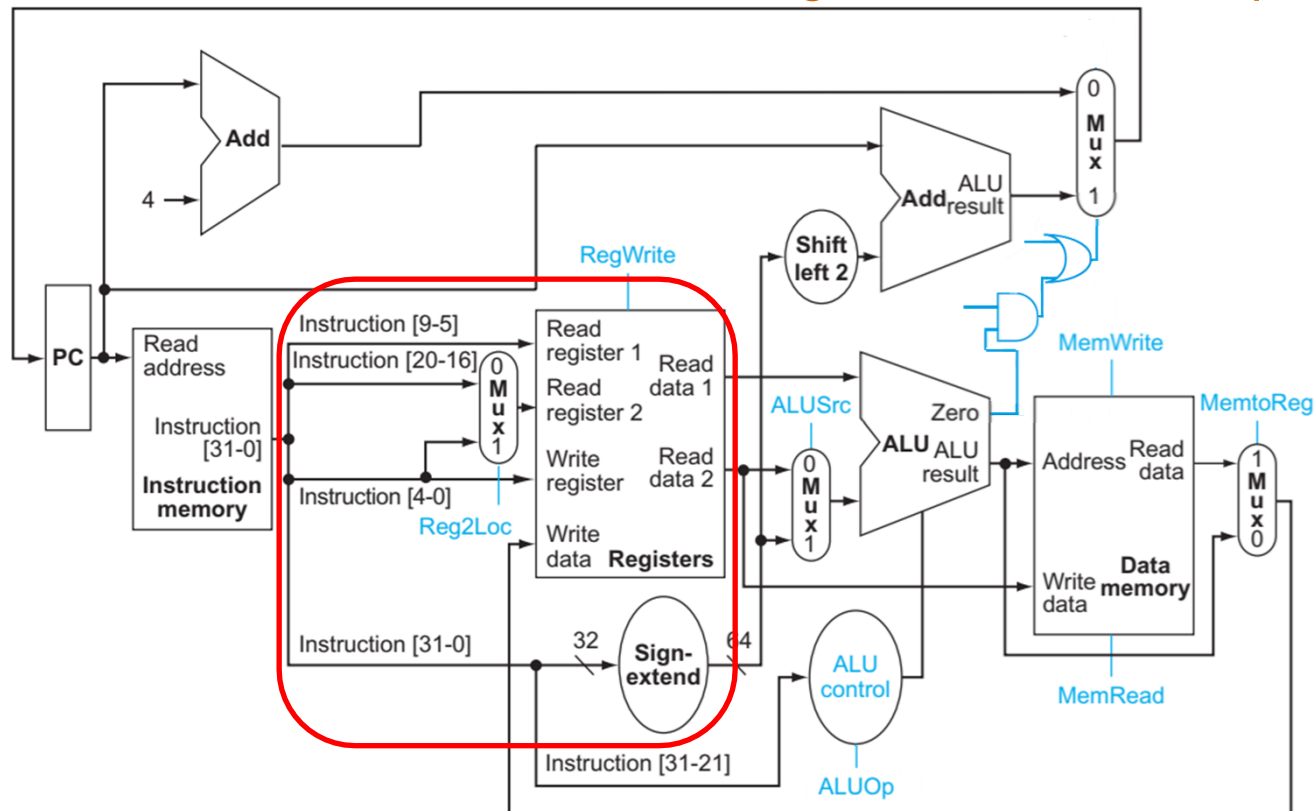
- Công đoạn 1: Nạp lệnh (Instruction Fetch).

- Nạp lệnh 32 bit từ bộ nhớ tại địa chỉ trong thanh ghi PC vào thanh ghi lệnh. Công đoạn này như nhau cho tất cả các lệnh.
- Sau đó, tăng PC để chuẩn bị nạp lệnh kế tiếp sau khi xử lý xong lệnh này ($PC = PC + 4$).



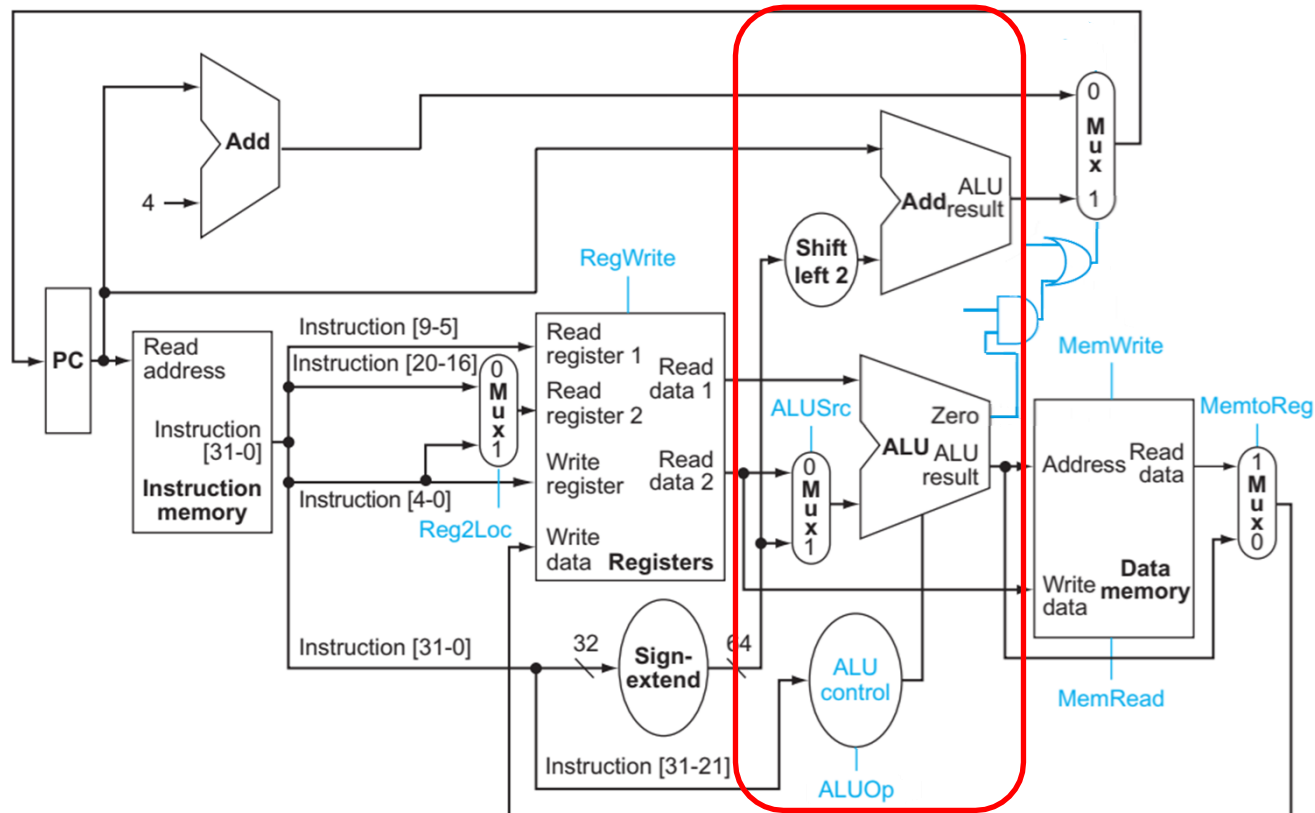
Các công đoạn lệnh (2/5)

- Công đoạn 2: Giải mã lệnh (Instruction Decode).
 - Phân tích các trường trong lệnh.
 - Opcode(bit 21-31), Rn(bit 5-9), Rm(bit 16-20), Rd(bit 0-4) .
 - Sau đó, đọc giá trị các thanh ghi nguồn, chuyển đổi các giá trị address để chuẩn bị thực hiện công đoạn tính toán tiếp theo.



Các công đoạn lệnh (3/5)

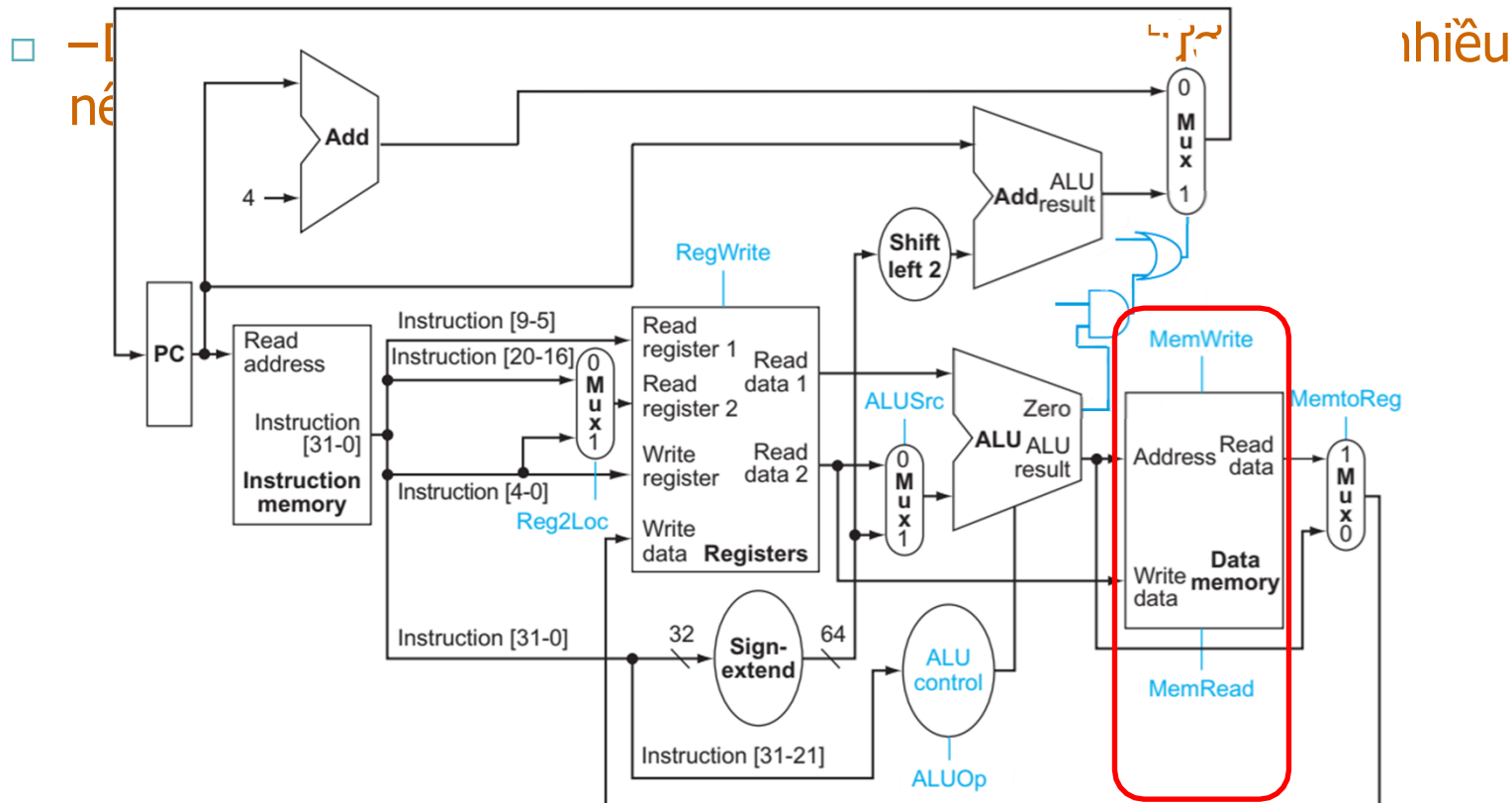
- Công đoạn 3: Tính toán (ALU – Arithmetic-Logic Unit).
 - Công việc chính của hầu hết các lệnh thực hiện tại công đoạn này: tính toán số học (ADD, SUB), luận lý (AND, ORR), so sánh (CBZ).
 - Lệnh CBZ, B tính vị trí cần nhảy tới: $PC = PC + (\text{address} * 4)$.
 - Lệnh LDUR và STUR tính địa chỉ bộ nhớ cần truy xuất $Rn + \text{address}$



Các công đoạn lệnh (4/5)

10 Công đoạn 4: Truy xuất bộ nhớ (Memory Access).

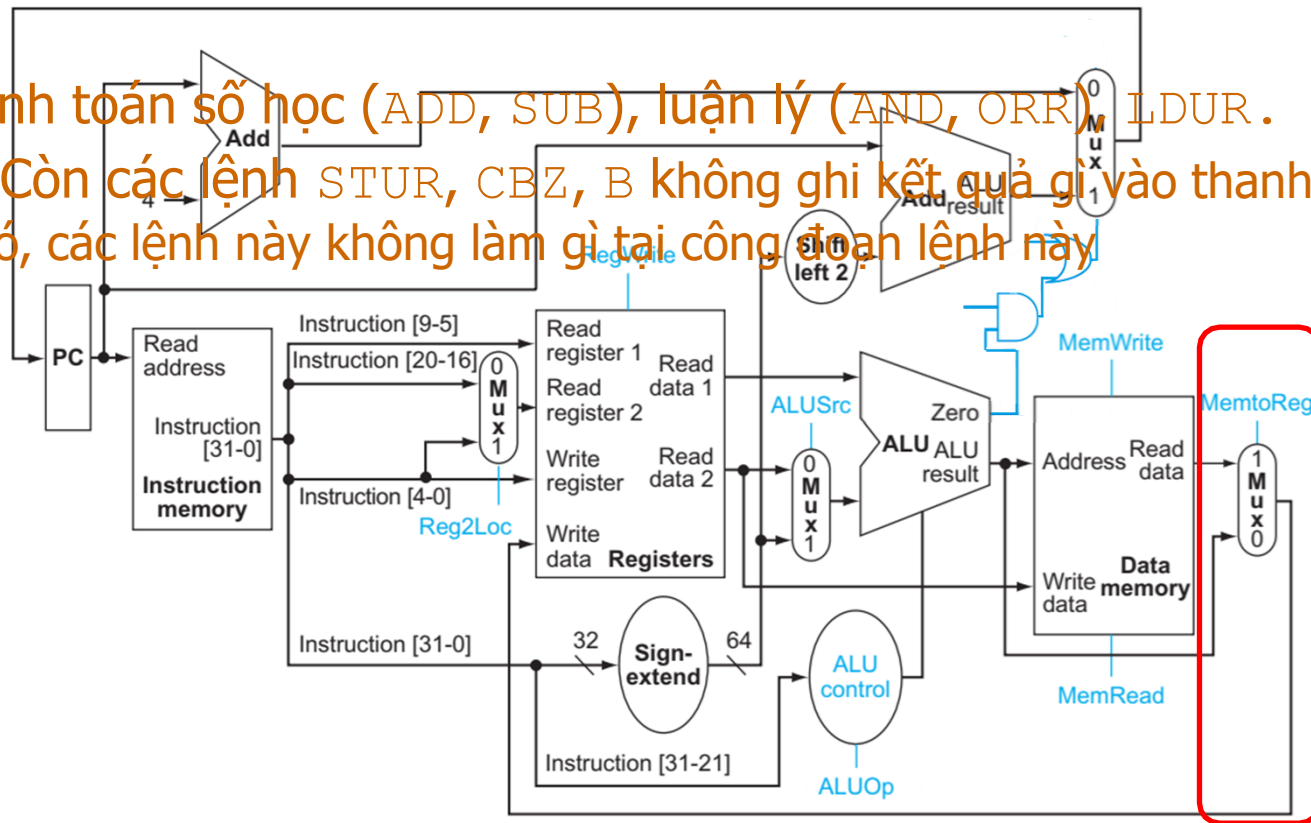
- Thực sự chỉ có lệnh LDUR và STUR thực hiện công đoạn lệnh này.



Các công đoạn lệnh (5/5)

10 Công đoạn 5: Ghi kết quả vào thanh ghi (Register Write).

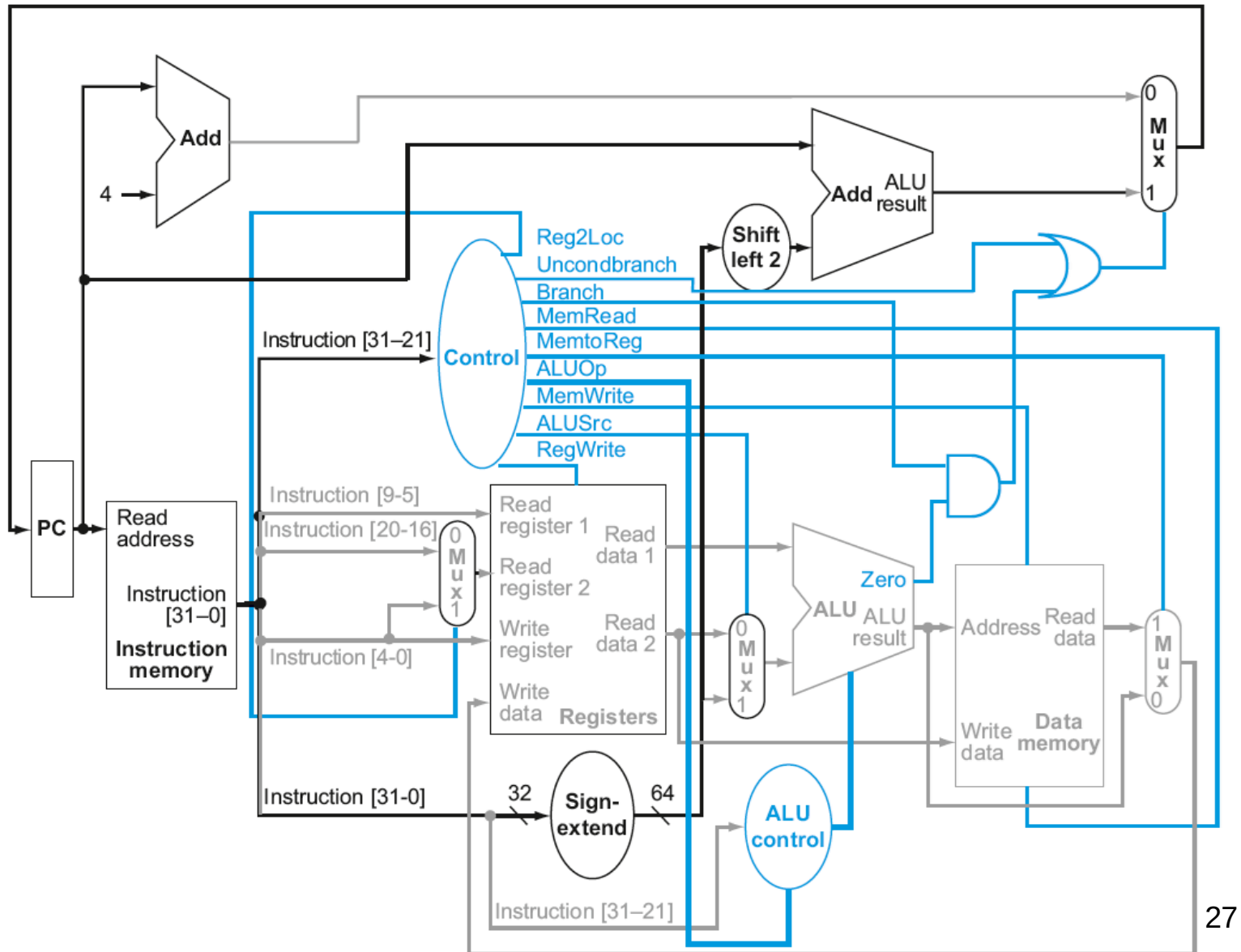
- Hầu hết các lệnh đều ghi kết quả tính toán vào một thanh ghi như tính toán số học (ADD, SUB), luận lý (AND, ORR) LDUR.
- Còn các lệnh STUR, CBZ, B không ghi kết quả gì vào thanh ghi. Do đó, các lệnh này không làm gì tại công đoạn lệnh này



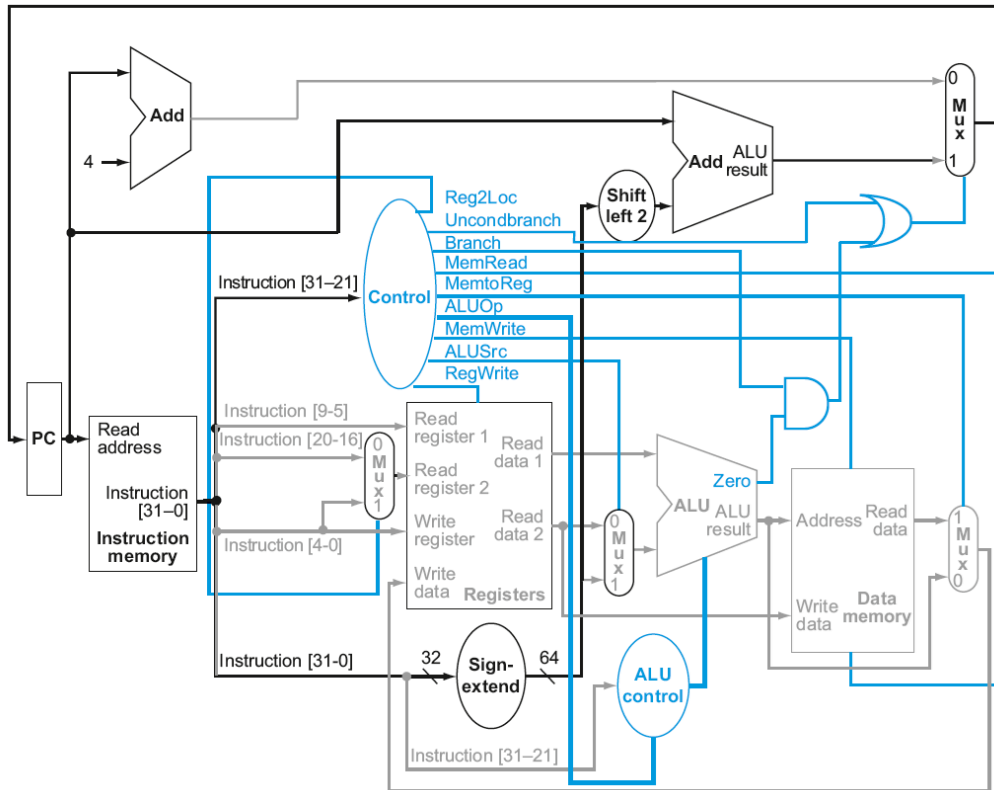
Tại sao lại 5 công đoạn ?

- Chỉ có lệnh `LDUR` thực hiện cả 5 công đoạn. Vậy tại sao `LEGV8` lại chia làm 5 công đoạn ?
 - Đó là sự tổ hợp đầy đủ cho tất cả các thao tác cần thiết của tất cả các lệnh.
 - Thời gian thực hiện mỗi công đoạn không quá chênh lệch nhau.
- Có thể có nhiều công đoạn lệnh hơn không ?
 - Có, các kiến trúc khác như x86.

Control cho tất cả lệnh LEGv8 thu gọn R-Format, LDUR, STUR, CBZ và B



Thiết kế đơn vị điều khiển ALU (1/2)



ALU control	Function
0000	AND
0001	OR
0010	add
0110	subtract
0111	pass input b

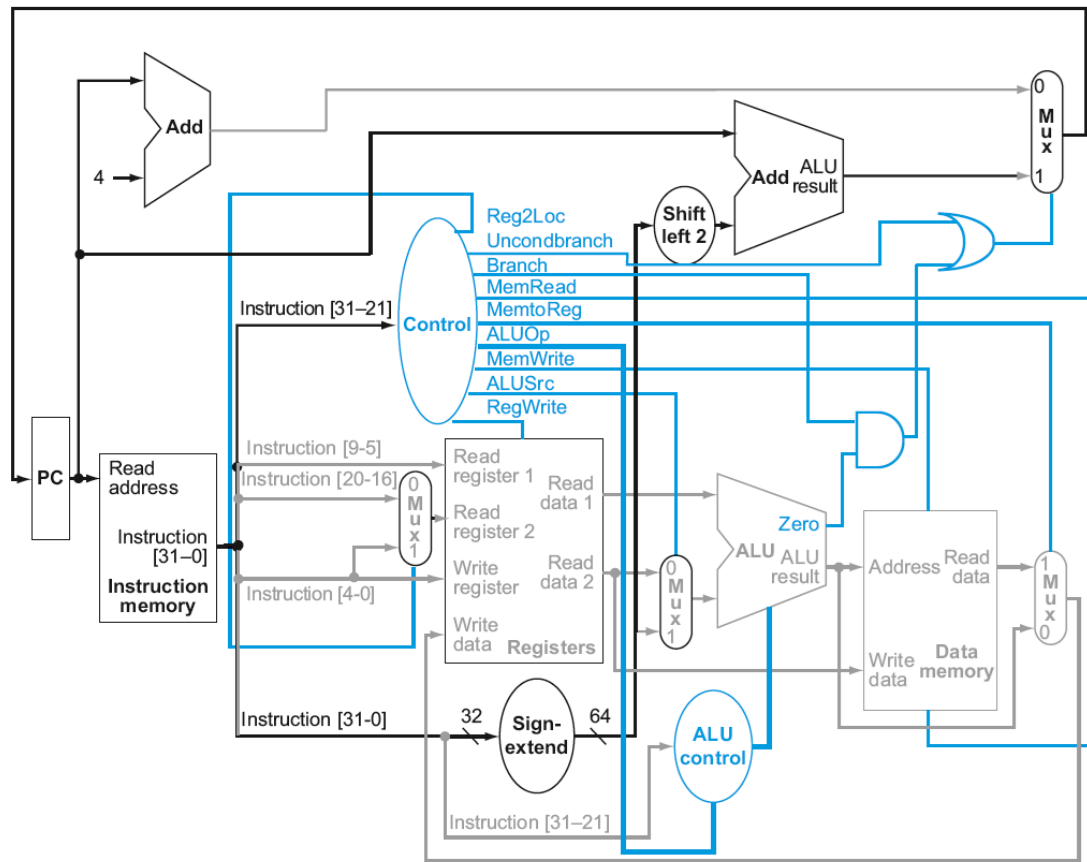
opcode	ALUOp	Operation	Opcode field	ALU function	ALU control
LDUR	00	load register	XXXXXXXXXXXX	add	0010
STUR	00	store register	XXXXXXXXXXXX	add	0010
CBZ	01	compare & branch on zero	XXXXXXXXXXXX	pass input b	0111
R-type	10	add	10001011000	add	0010
		subtract	11001011000	subtract	0110
		AND	10001010000	AND	0000
		ORR	10101010000	OR	0001

Thiết kế đơn vị điều khiển ALU (2/2)

opcode	ALUOp	Operation	Opcode field	ALU function	ALU control
LDUR	00	load register	XXXXXXXXXXXX	add	0010
STUR	00	store register	XXXXXXXXXXXX	add	0010
CBZ	01	compare and branch on zero	XXXXXXXXXXXX	pass input b	0111
R-type	10	add	10001011000	add	0010
		subtract	11001011000	subtract	0110
		AND	10001010000	AND	0000
		ORR	10101010000	OR	0001

ALUOp		Opcode field											Operation
ALUOp1	ALUOp0	I[31]	I[30]	I[29]	I[28]	I[27]	I[26]	I[25]	I[24]	I[23]	I[22]	I[21]	
0	0	X	X	X	X	X	X	X	X	X	X	X	0010
X	1	X	X	X	X	X	X	X	X	X	X	X	0111
1	X	1	0	0	0	1	0	1	1	0	0	0	0010
1	X	1	1	0	0	1	0	1	1	0	0	0	0110
1	X	1	0	0	0	1	0	1	0	0	0	0	0000
1	X	1	0	1	0	1	0	1	0	0	0	0	0001

Thiết kế đơn vị điều khiển chính (1/2)



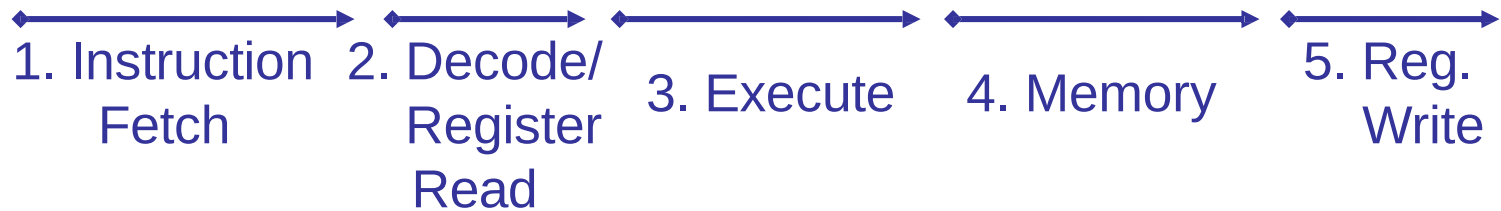
Instruction	Reg 2Loc	ALU Src	Mem toReg	Reg Write	Mem Read	Mem Write	Branch	Uncond Branch	ALU Op1	ALU Op0
R-Format	0	0	0	1	0	0	0	0	1	0
LDUR	X	1	1	1	1	0	0	0	0	0
STUR	1	1	X	0	0	1	0	0	0	0
CBZ	1	0	X	0	0	0	1	0	0	1
B	X	X	X	0	0	0	0	1	X	X

Thiết kế đơn vị điều khiển chính (2/2)

Input or Output	Signal name	R-Format	LDUR	STUR	CBZ	B
Input	I[31]	1	1	1	1	0
	I[30]	X	1	1	0	0
	I[29]	X	1	1	1	0
	I[28]	0	1	1	1	1
	I[27]	1	1	1	0	0
	I[26]	0	0	0	1	1
	I[25]	1	0	0	0	X
	I[24]	X	0	0	0	X
	I[23]	0	0	0	X	X
	I[22]	0	1	0	X	X
	I[21]	0	0	0	X	X
Output	Reg2Loc	0	X	1	1	X
	ALUSrc	0	1	1	0	X
	MemtoReg	0	1	X	X	X
	RegWrite	1	1	0	0	0
	MemRead	0	1	0	0	0
	MemWrite	0	0	1	0	0
	Branch	0	0	0	1	0
	UncondBranch	0	0	0	0	1
	ALUOp1	1	0	0	0	X
	ALUOp0	0	0	0	1	X

Kỹ thuật thiết kế CPU một chu kỳ

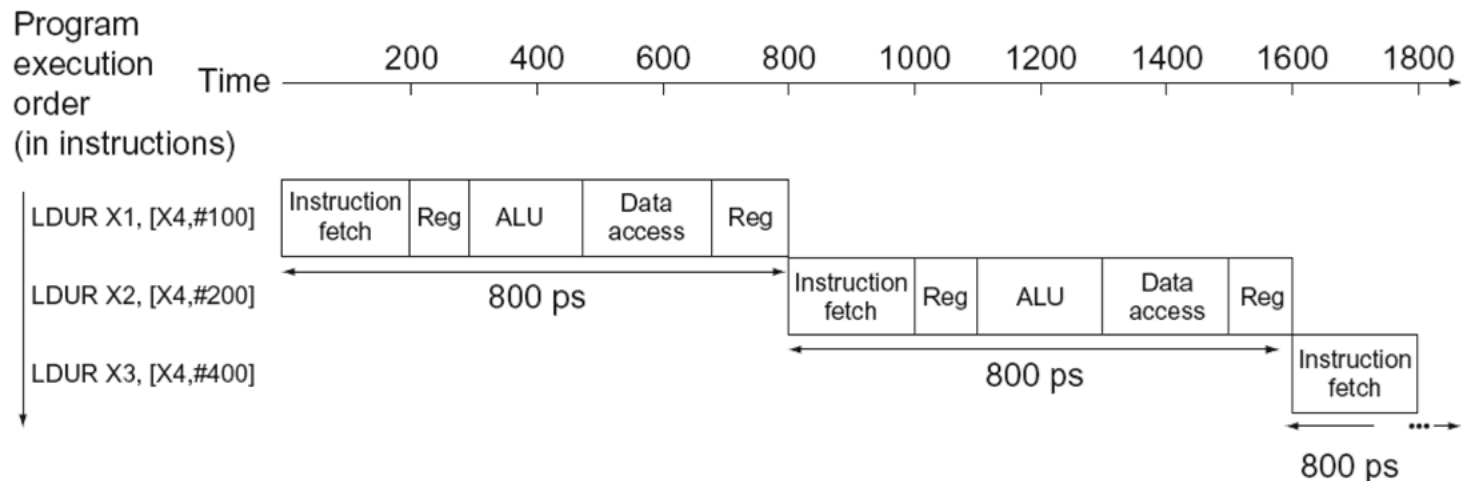
- Thiết kế CPU 1 chu kỳ: Tất cả các công đoạn của 1 lệnh được xử lý trong 1 chu kỳ đồng hồ.
 - Chu kỳ đồng hồ phải đủ lâu để có thể hoàn thành xử lý mọi lệnh.



Instruction	Instruction fetch	Register read	ALU op	Memory access	Register write	Total time
LDUR	200ps	100 ps	200ps	200ps	100 ps	800ps
STUR	200ps	100 ps	200ps	200ps		700ps
R-format	200ps	100 ps	200ps		100 ps	600ps
CBZ	200ps	100 ps	200ps			500ps
B	200ps	100 ps	200ps			500ps

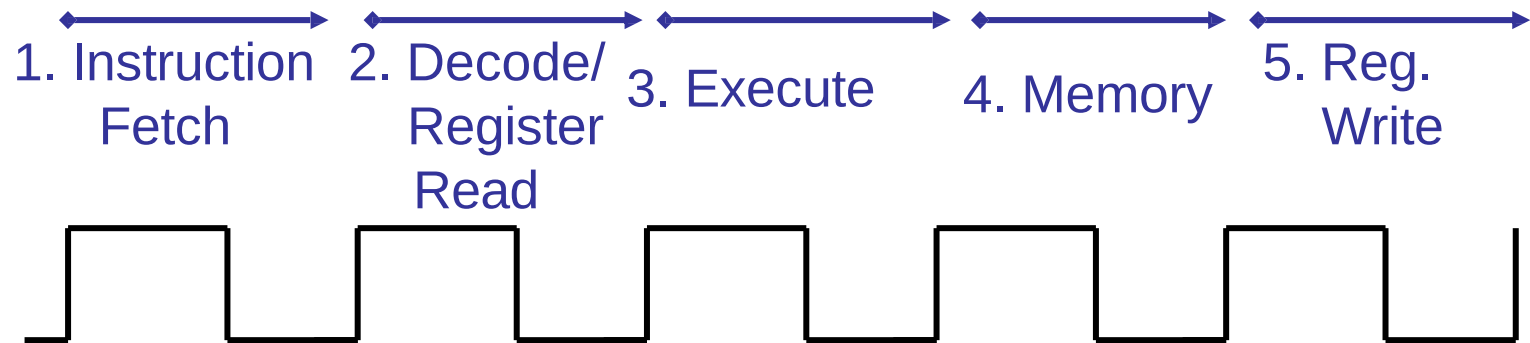
Hạn chế của kỹ thuật thiết kế CPU một chu kỳ

- Kỹ thuật CPU 1 chu kỳ không còn được sử dụng vì không hiệu quả.
 - Tất cả công đoạn của 1 lệnh phải xử lý trong một chu kỳ theo tín hiệu đồng bộ nên các thành phần mạch có khả năng dùng chung đều được tách riêng, làm cho sơ đồ mạch phức tạp hơn.
 - Thành phần tính toán: ALU, Adder.
 - Thành phần lưu trữ: Instruction memory, Data memory.
 - Một chu kỳ đồng hồ phải đủ lâu để xử lý được lệnh phức tạp nhất. Như trong LEGv8 thu gọn, lệnh `LDUR` xử lý phức tạp nhất (5 công đoạn), trong khi các lệnh khác chỉ mất 3 (`CBZ`, `B`) hoặc 4 (`R-Format`, `STUR`) công đoạn.
 - Với chương trình có số lệnh cần thực thi là `IC` (instruction count) thì sẽ xử lý trong $800 \times IC$ (ps).



Kỹ thuật thiết kế CPU nhiều chu kỳ

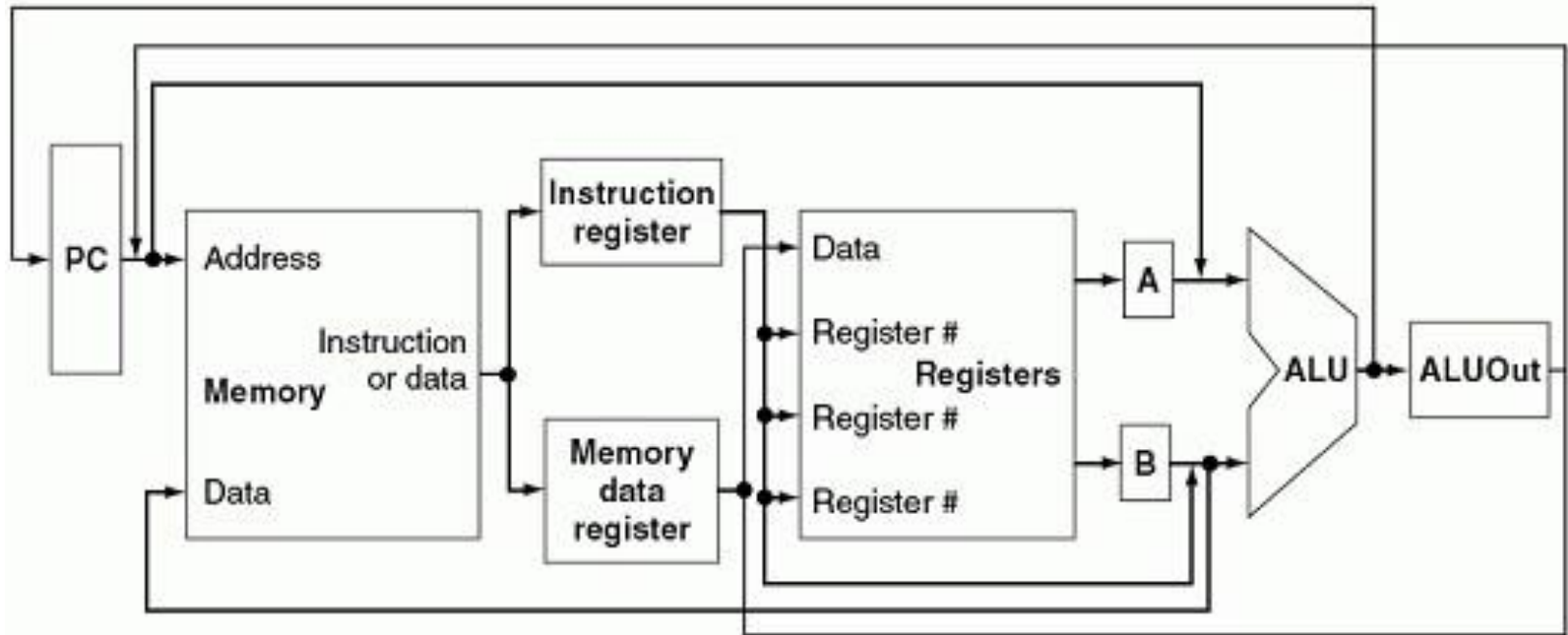
- Thiết kế CPU nhiều chu kỳ: Mỗi công đoạn lệnh thực hiện trong 1 chu kỳ.
 - Mỗi chu kỳ đồng hồ phải đủ lâu để thực hiện mọi công đoạn lệnh.



Instruction	Instruction fetch	Register read	ALU op	Memory access	Register write	Total time
LDUR	200ps	200 ps	200ps	200ps	200 ps	1000ps
STUR	200ps	200 ps	200ps	200ps		800ps
R-format	200ps	200 ps	200ps		200 ps	800ps
CBZ	200ps	200 ps	200ps			600ps
B	200ps	200 ps	200ps			600ps

Sơ đồ khối CPU nhiều chu kỳ

- Do mỗi công đoạn được thực thi trong một chu kỳ riêng, nên có thể ghép các thành phần mạch dùng chung (ALU + Adder, Imem + DMem) mà không xảy ra ùng độ
- Cần thêm các thanh ghi để lưu giữ kết quả trung gian của các công đoạn lệnh (A, B, ALUOut,...)



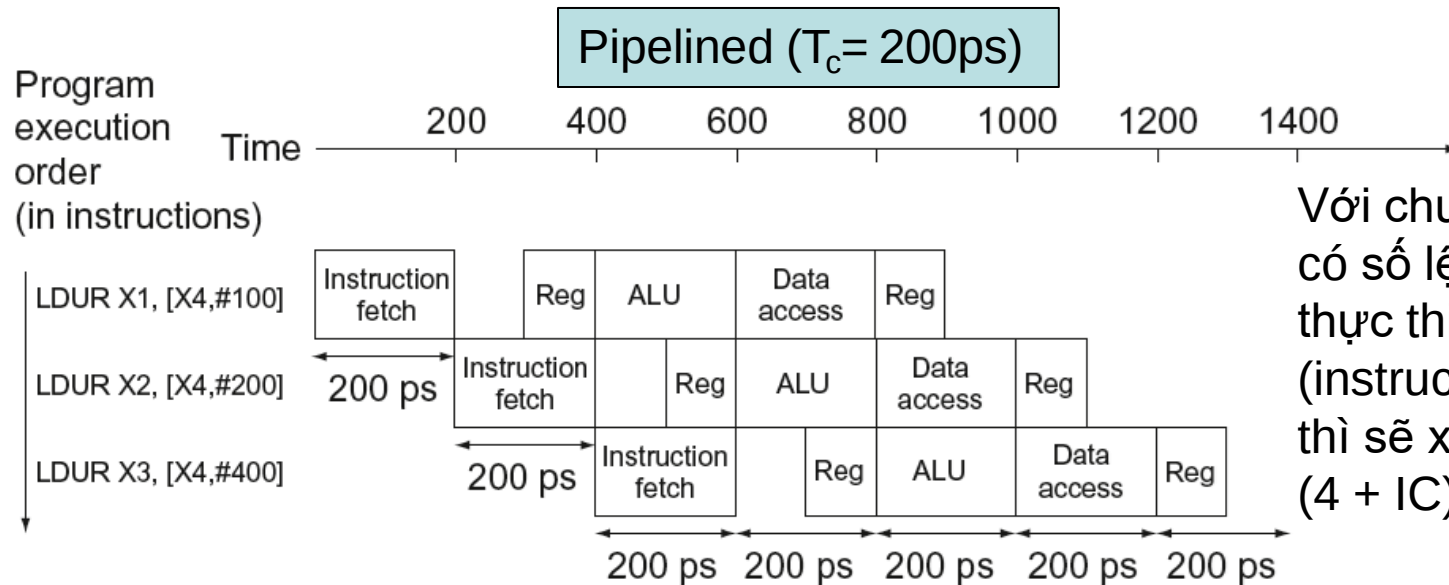
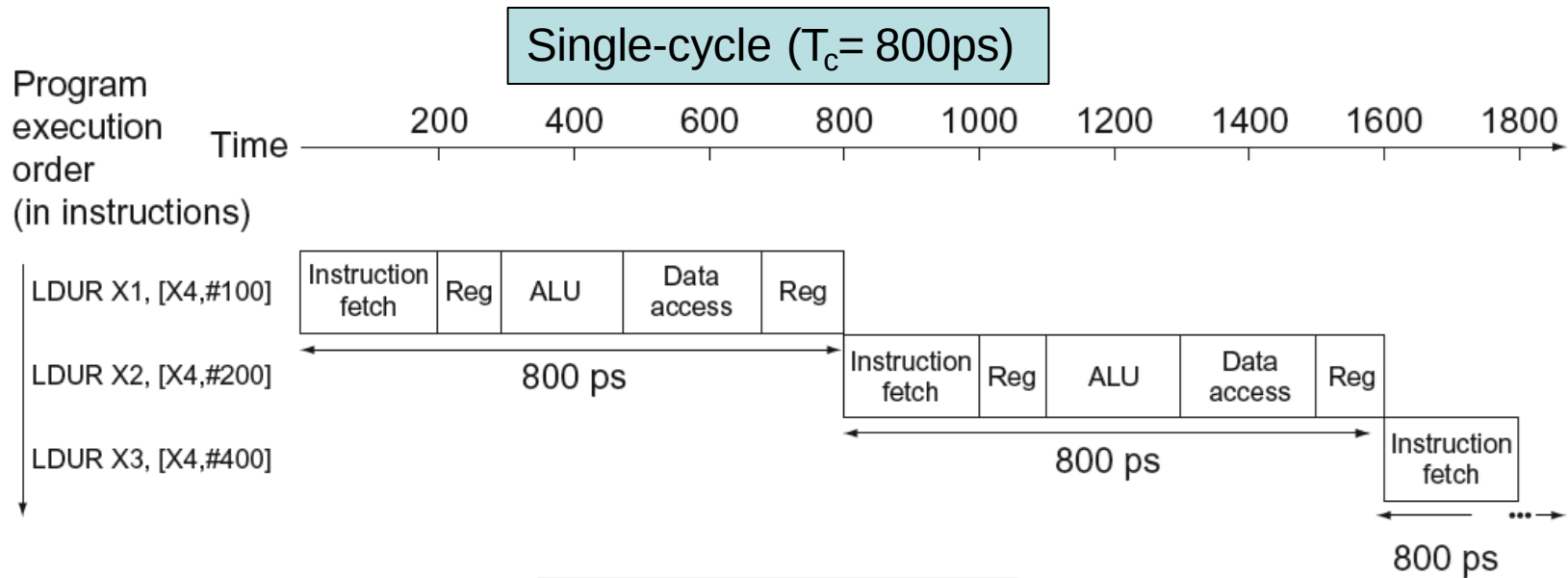
Hiệu năng của kỹ thuật thiết kế CPU nhiều chu kỳ

- Kỹ thuật thiết kế CPU nhiều chu kỳ có mang lại hiệu năng tốt hơn so với Kỹ thuật thiết kế CPU nhiều chu kỳ không ?
- Theo thống kê:

	Load	Store	ALU	Branch	Jump
Cycles	5	4	4	3	3
Frequency(%)	25	10	52	11	2

- Với chương trình có số lệnh cần thực thi là IC (instruction count) thì sẽ xử lý trong
$$(0.25 \times 5 + 0.1 \times 4 + 0.52 \times 4 + 0.11 \times 3 + 0.02 \times 3) \times 200 \times IC$$
$$= 824 \times IC \text{ (ps) !!!}$$

Kỹ thuật Pipelining



Với chương trình có số lệnh cần thực thi là IC (instruction count) thì sẽ xử lý trong $(4 + IC) \times 200 \text{ (ps)}$.

Các vấn đề cần giải quyết của kỹ thuật Pipelining (1/2)

- Structure hazards.

- Khi các công đoạn của các lệnh được nạp chồng, trong 1 chu kỳ có thể sẽ xảy ra đụng độ việc sử dụng các thành phần mạch như: ALU, Mem, Register.
- Giải pháp:
 - Separate logic circuit components.

- Data hazard.

- Lệnh sau có thể phụ thuộc dữ liệu lệnh trước. Việc nạp chồng thực hiện có thể xảy ra trường hợp dữ liệu kết quả của lệnh trước chưa có mà lệnh sau đã cần đọc dữ liệu này.
- Giải pháp:
 - Forwarding.
 - Stalling.
 - Code scheduling.

Các vấn đề cần giải quyết của kỹ thuật Pipelining (2/2)

- Control hazard.

- Khi đang thực hiện lệnh rẽ nhánh thì không thể biết lệnh tiếp theo sẽ được thực hiện là lệnh nào. Việc nạp chồng có thể sẽ không hiệu quả.
- Giải pháp:
 - Branch stalling.
 - Delayed Branch.
 - Static branch prediction.
 - Predict forward branches not taken.
 - Predict backward branches taken.
 - Dynamic branch prediction.
 - Branch History Table.
 - Correlated Branch Prediction.
 - Tournament Predictors.
 - Branch Target Buffer.

Các kỹ thuật hiện đại nâng cao hiệu năng (1/2)

- Uniprocessor, multiprocessors, sequential programming, parallel programming and parallelism.
 - Sequential/concurrent software can run on serial/parallel hardware
 - Parallel processing program, parallel software

		Data Streams	
		Single	Multiple
Instruction Streams	Single	SISD : Intel Pentium 4	SIMD : x86's SSE instructions
	Multiple	MISD : No examples today	MIMD : Intel Xeon e5345

- Instruction-Level Parallelism.
 - Pipelining.
 - Superscalar.
 - Speculation.
 - Vector.
 - Multimedia extension (MMX, SSE, SSE2, AVX, AVX2).

Các kỹ thuật hiện đại nâng cao hiệu năng (1/2)

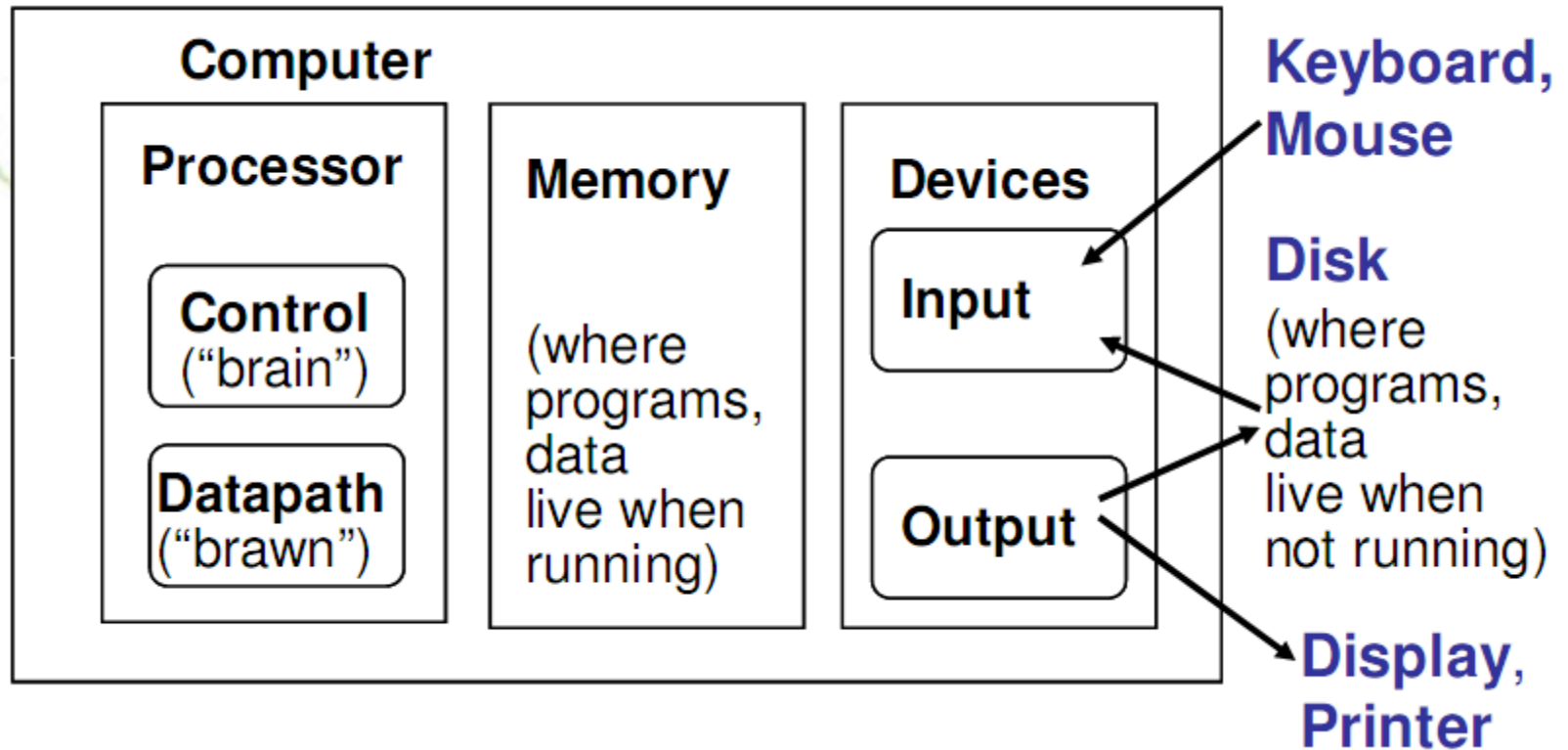
- Hardware Multithreading.
 - Fine-grain multithreading.
 - Coarse-grain multithreading.
 - Simultaneous Multithreading (SMT).
- Multicore, Shared Memory Multiprocessors / Symmetric Multiprocessors (SMP).
 - Uniform memory access (UMA) multiprocessors.
 - Non-uniform memory access (NUMA) multiprocessors.
 - Graphics Processing Units (GPU).
- Message-Passing Multiprocessors.
 - Clusters.
 - Warehouse-Scale Computers (Cloud).
 - Grid computing.

Tham khảo

- Chương 4 & Chương 6
 - David A. Patterson, John L. Hennessy. (2016). ***Computer Organization and Design ARM Edition: The Hardware Software Interface*** (1st ed.). Oxford: Morgan Kaufmann.

5 thành phần cơ bản của máy tính

43



Bộ vi xử lý (CPU)

44

- Datapath
 - ▣ Registers
 - ▣ ALU
- Control unit
- **Stalling:** CPU = {Registers, ALU, Control unit, Internal bus}

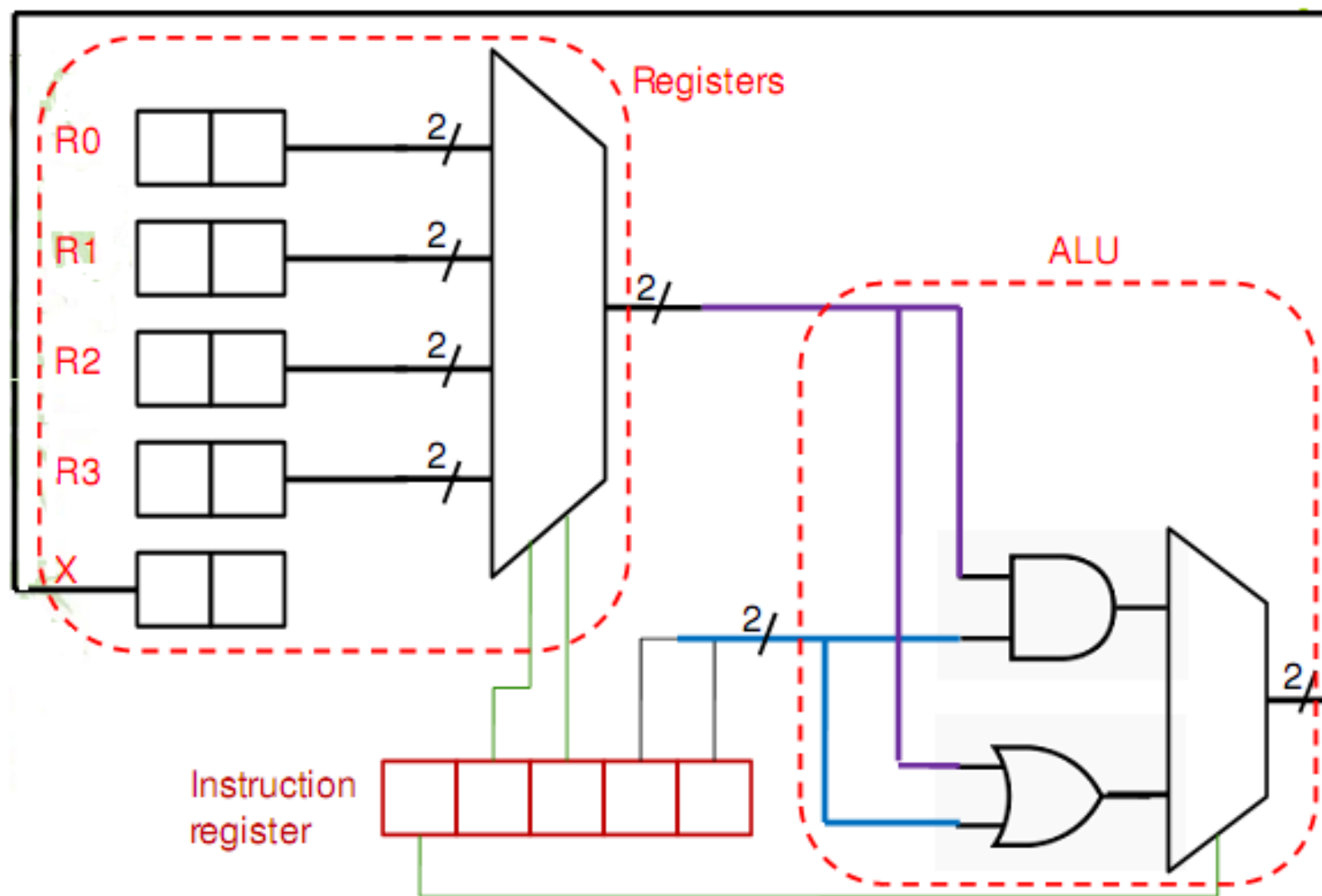
AND R, value
OR R, value

op	id	value
----	----	-------

$X = R_{id} \text{ op value}$

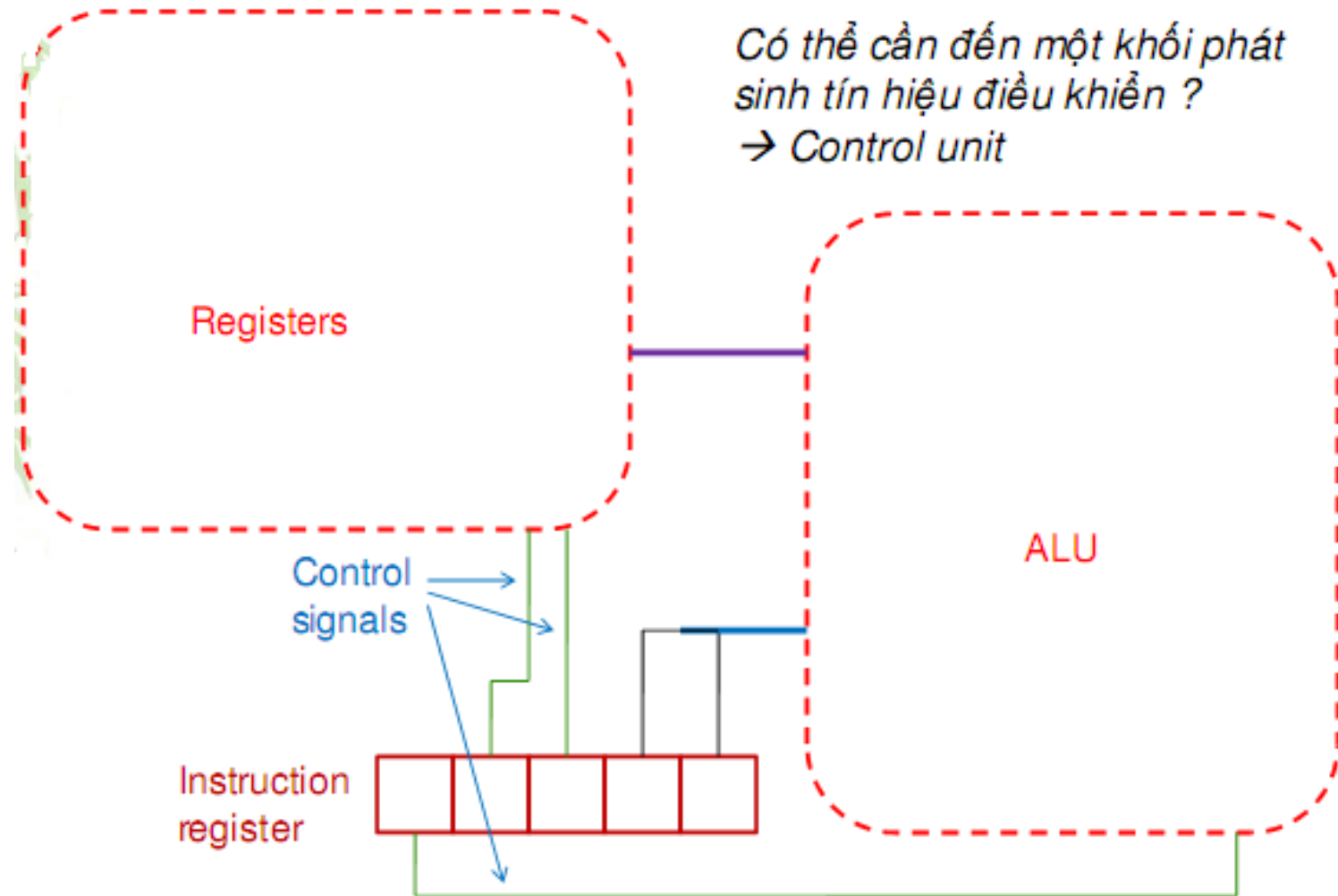
1	0	1	1	0
---	---	---	---	---

$X = R1 \text{ OR } 2$



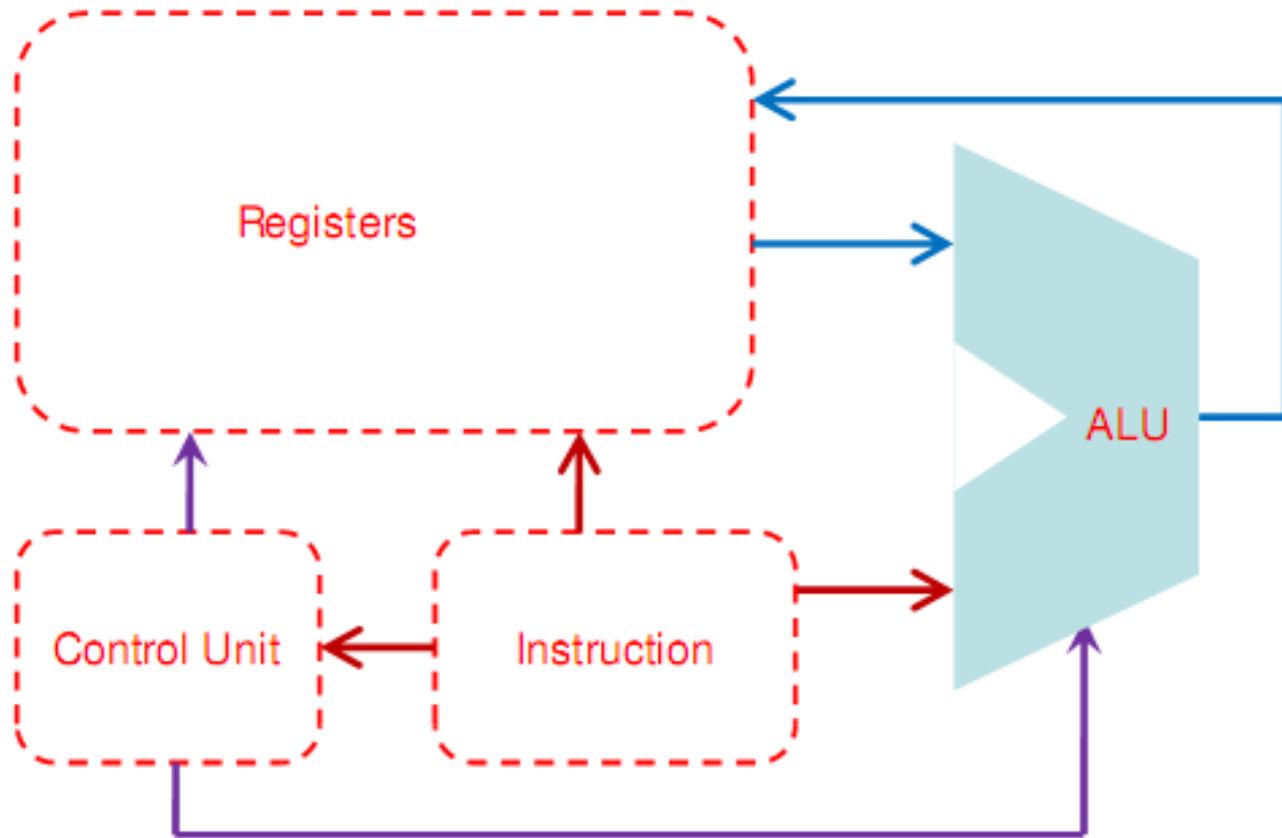
Control signals

46



Datapath & Control unit

47



MIPS thu gọn

48

- Lệnh truy xuất bộ nhớ: **lw, sw**
 - Lệnh số học – luận lý: **add, sub, and, or, slt**
 - Lệnh rẽ nhánh: **beq, j**
- Thiết kế bộ xử lý (Datapath và Control) cho tập lệnh MIPS thu gọn này ?

- Bất kỳ câu lệnh nào muốn thực thi cũng phải qua 2 bước đầu tiên:
 - Gửi địa chỉ lệnh chứa trong thanh ghi PC (Program counter) đến bộ nhớ lệnh để lấy nội dung câu lệnh từ bộ nhớ
 - Xác định toán hạng trong câu lệnh → Đọc các thanh ghi chứa toán hạng có địa chỉ tương ứng
- Các bước tiếp theo phụ thuộc vào từng nhóm lệnh khác nhau
- Tập lệnh MIPS thu gọn có các bước thực thi giống nhau ở khá nhiều điểm, khác biệt chủ yếu nằm ở các bước thực thi cuối của câu lệnh

Instruction format

50

Field	0	rs	rt	rd	shamt	funct
Bit positions	31:26	25:21	20:16	15:11	10:6	5:0

a. R-type instruction

Field	35 or 43	rs	rt	address
Bit positions	31:26	25:21	20:16	15:0

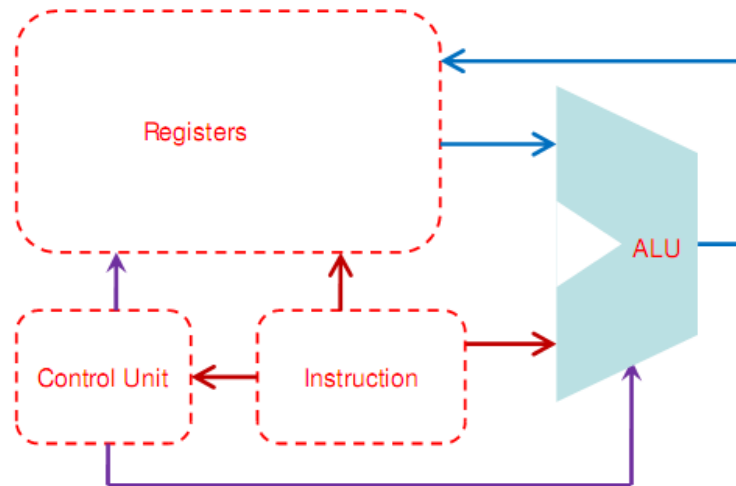
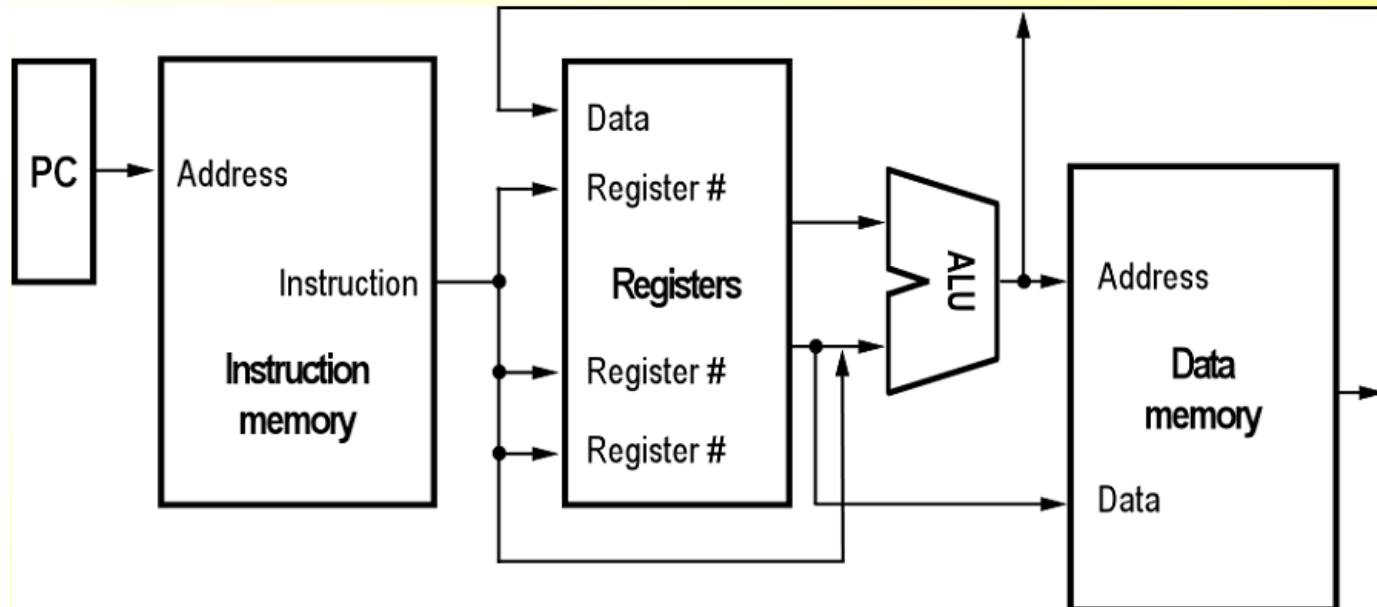
b. Load or store instruction

Field	4	rs	rt	address
Bit positions	31:26	25:21	20:16	15:0

c. Branch instruction

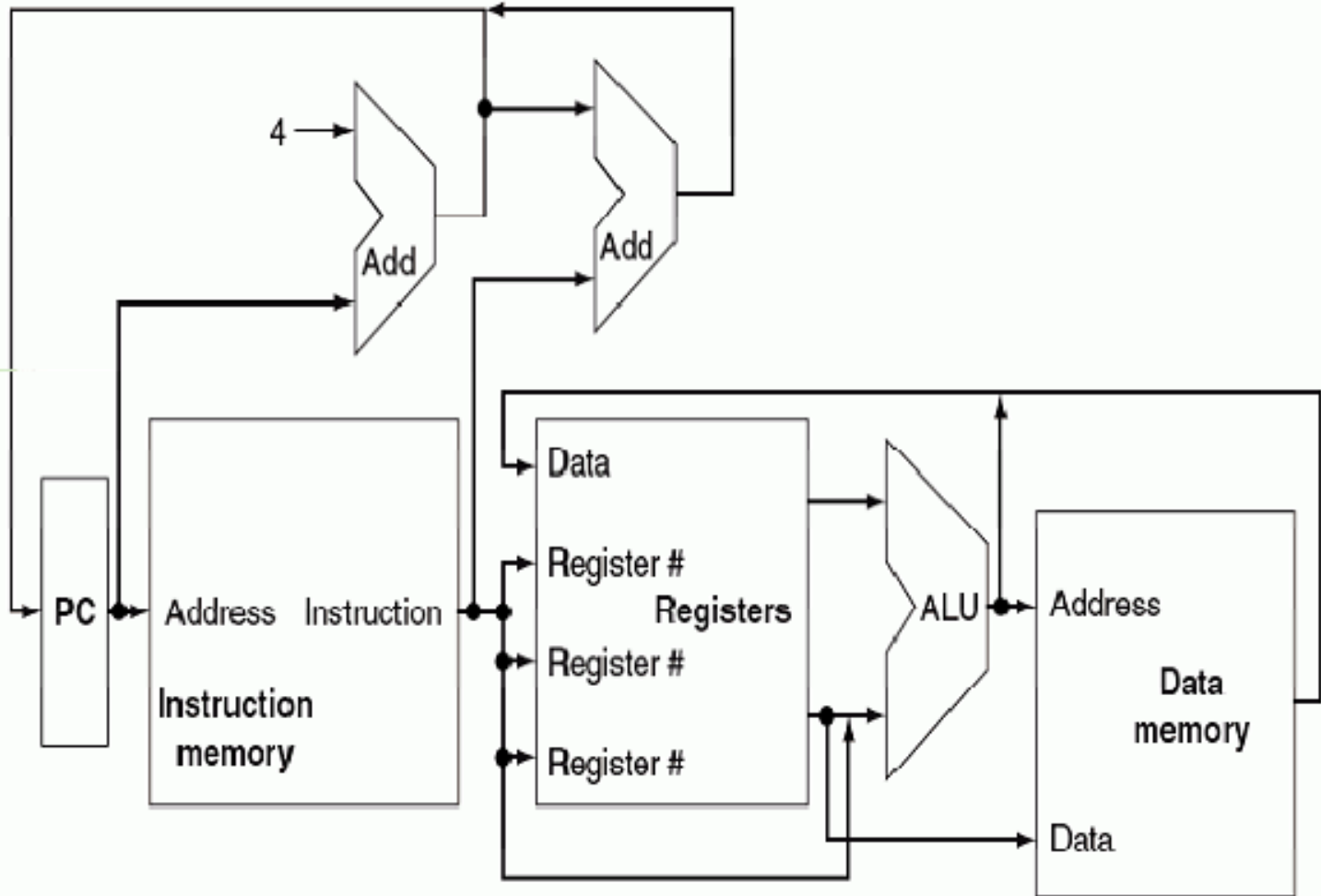
Sơ đồ thực thi tổng quát

51



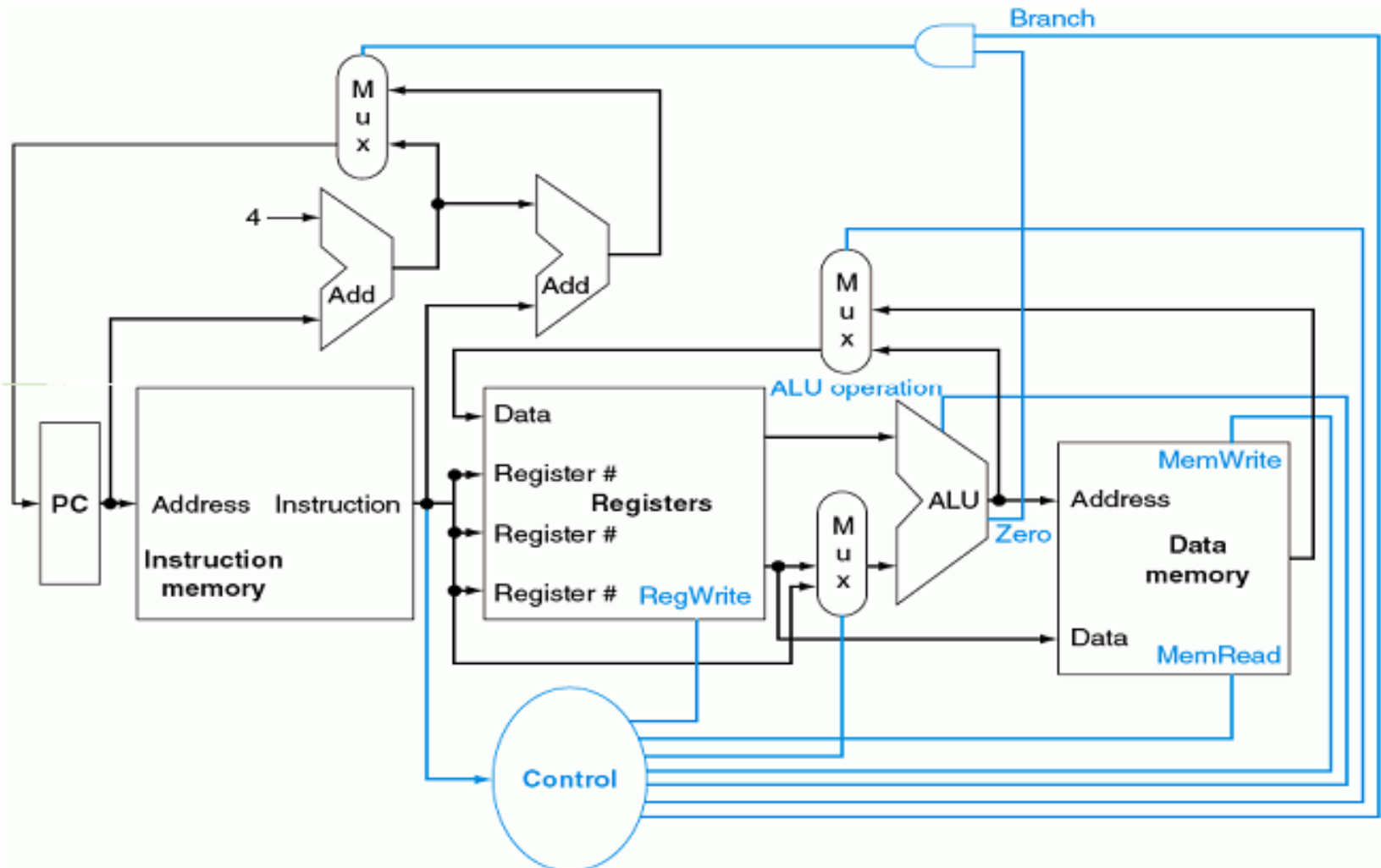
Dịch chuyển lệnh tiếp theo...

52



Sử dụng MUX để điều khiển

53

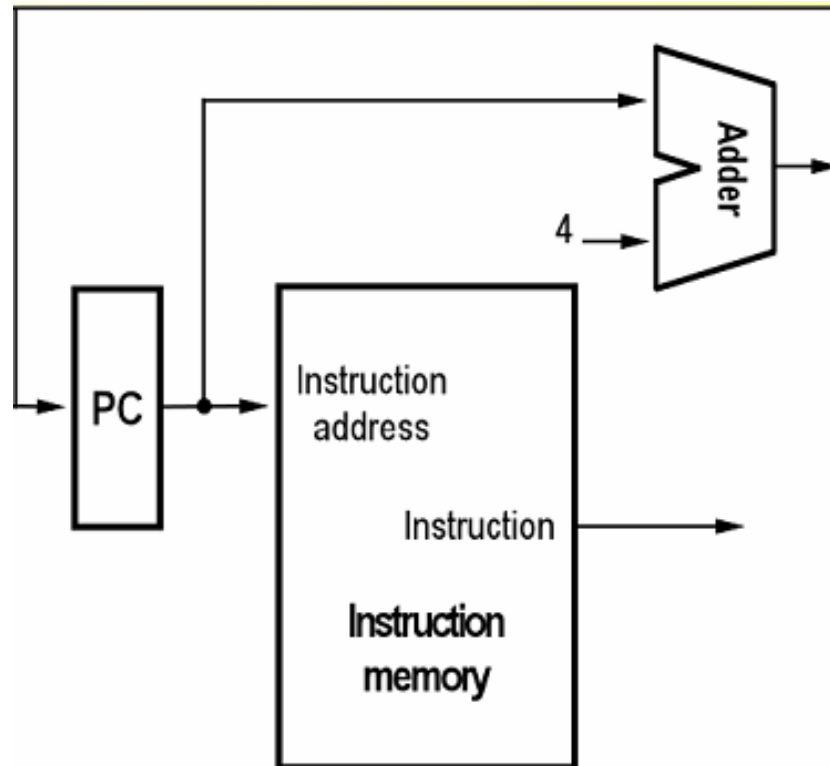


Xây dựng đường đi dữ liệu (Datapath)

54

- Phương thức xây dựng Datapath:
 - ▣ Xác định **kiến trúc của các phần tử cần thiết** cho câu lệnh
 - ▣ Xây dựng dần các **phân khúc** cho Datapath ứng với **từng công đoạn** trong thực thi câu lệnh
 - ▣ Tiến đến xây dựng **hoàn chỉnh** Datapath cho câu lệnh

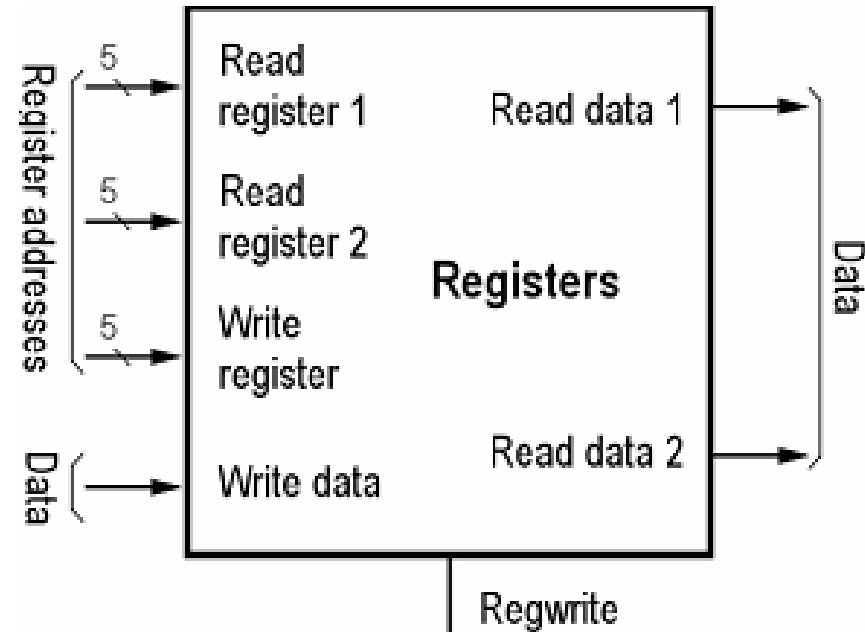
□ Dịch chuyển lệnh:



Kiến trúc các phần tử cần thiết

56

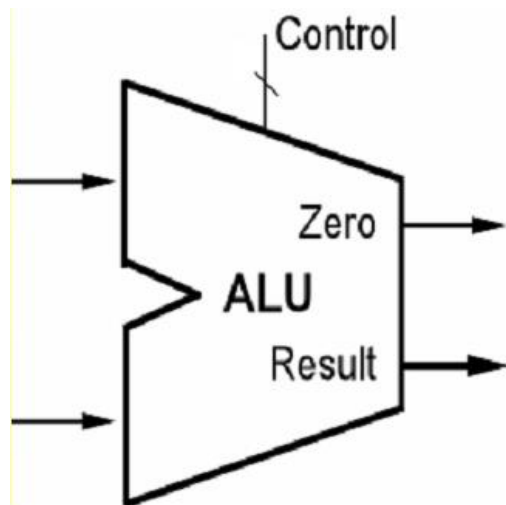
- Tập thanh ghi (register files)
 - 3 ngõ nhận địa chỉ thanh ghi
 - 1 ngõ ghi dữ liệu
 - 2 ngõ đọc dữ liệu (output)
 - 1 tín hiệu điều khiển ghi



Kiến trúc các phần tử cần thiết

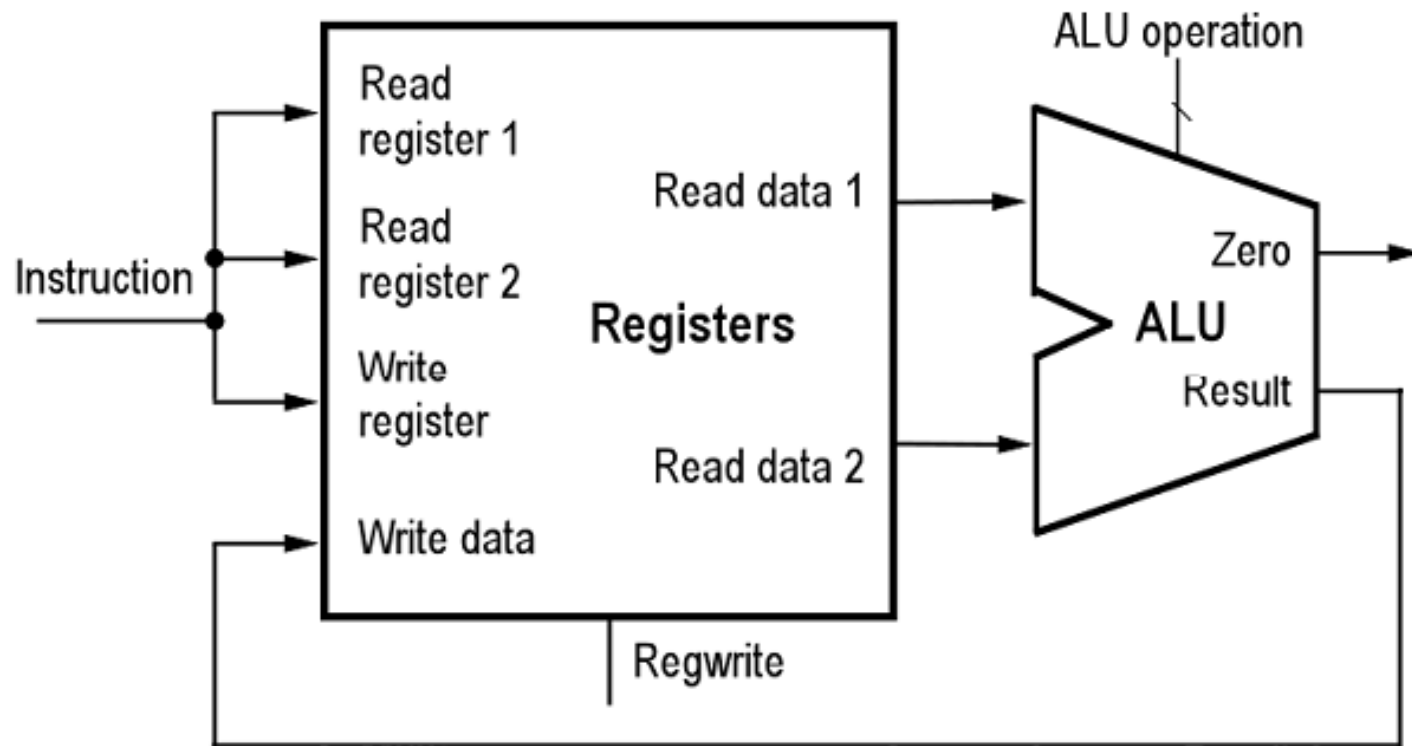
57

- Đơn vị số học – luận lý (ALU – Arithmetic Logic Unit)
 - 2 ngõ vào toán hạng (32-bit)
 - 1 ngõ ra kết quả (32 bit) và 1 bit zero (để chứa kết quả so sánh bằng)
 - 1 tín hiệu điều khiển (4 bit)



Register + ALU

58



Datapath cho I,J-format (lw, sw, beq, j) ?

59

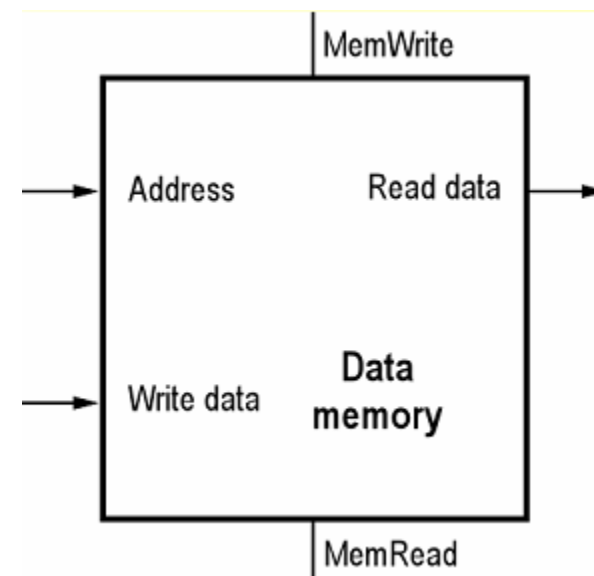
- Cần thêm 2 thành phần cơ bản:
 - ▣ Bộ nhớ dữ liệu (Data memory unit)
 - ▣ Bộ mở rộng dấu (Sign extended unit)

Datapath cho I,J-format (lw, sw, beq, j)

60

□ Bộ nhớ dữ liệu (Data memory unit)

- 1 ngõ nhận địa chỉ ô nhớ
- 1 ngõ nhận dữ liệu cần ghi
- 1 ngõ dữ liệu đọc (output)
- 2 tín hiệu điều khiển đọc / ghi



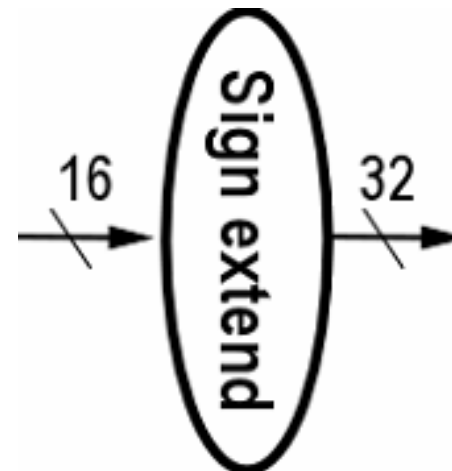
Datapath cho I-format (lw, sw, beq)

61

- Bộ mở rộng dấu (Sign extended unit)

- 1 ngõ nhập dữ liệu 16-bit

- 1 ngõ ra dữ liệu 32-bit

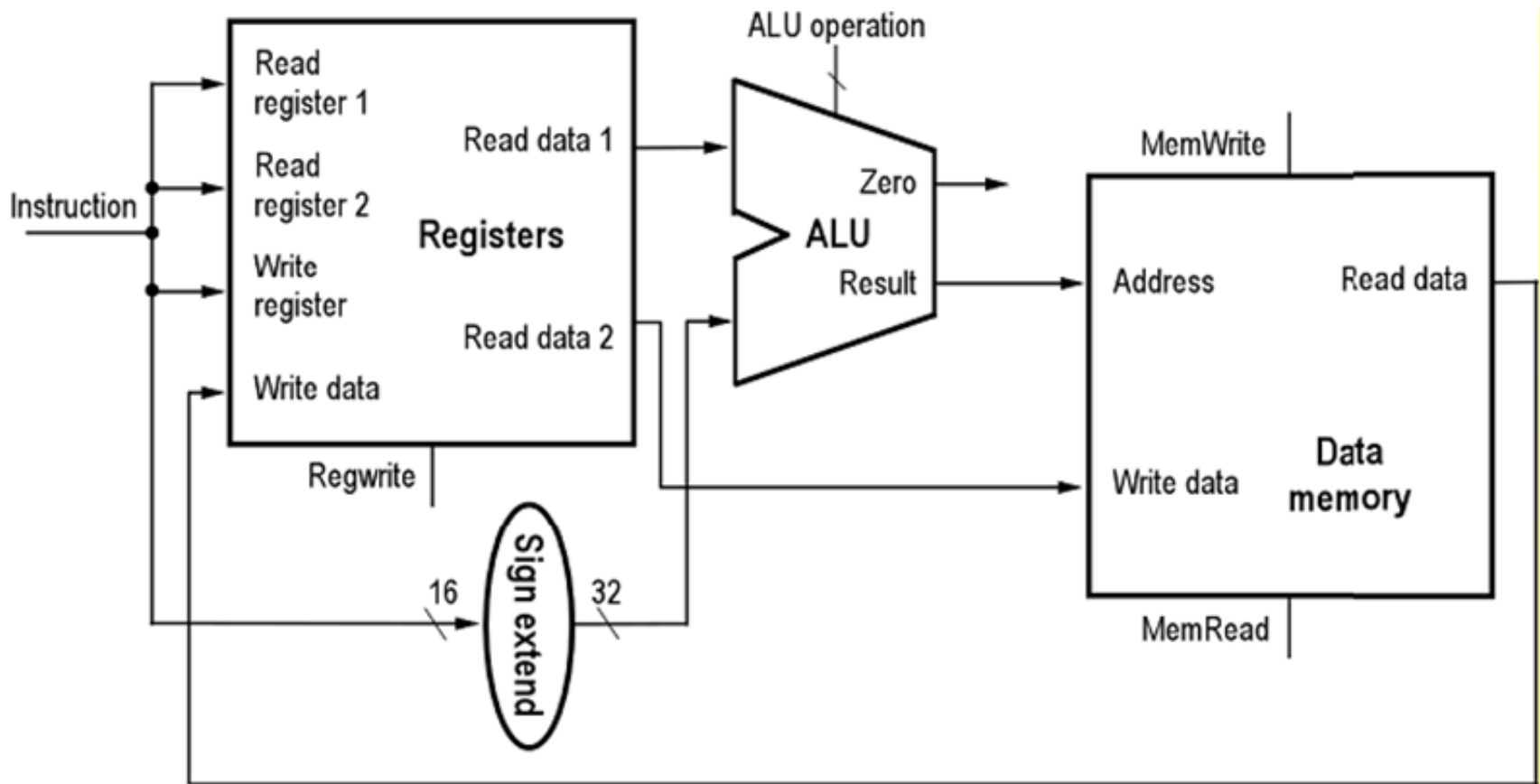


- lw \$s1, 4(\$s0) → 4: 16 bit → 04: 32 bit (sign-extended)

- beq \$s0, \$s1, target_label → target-label: 16 bit →
target-label: 32 bit (sign-extended)

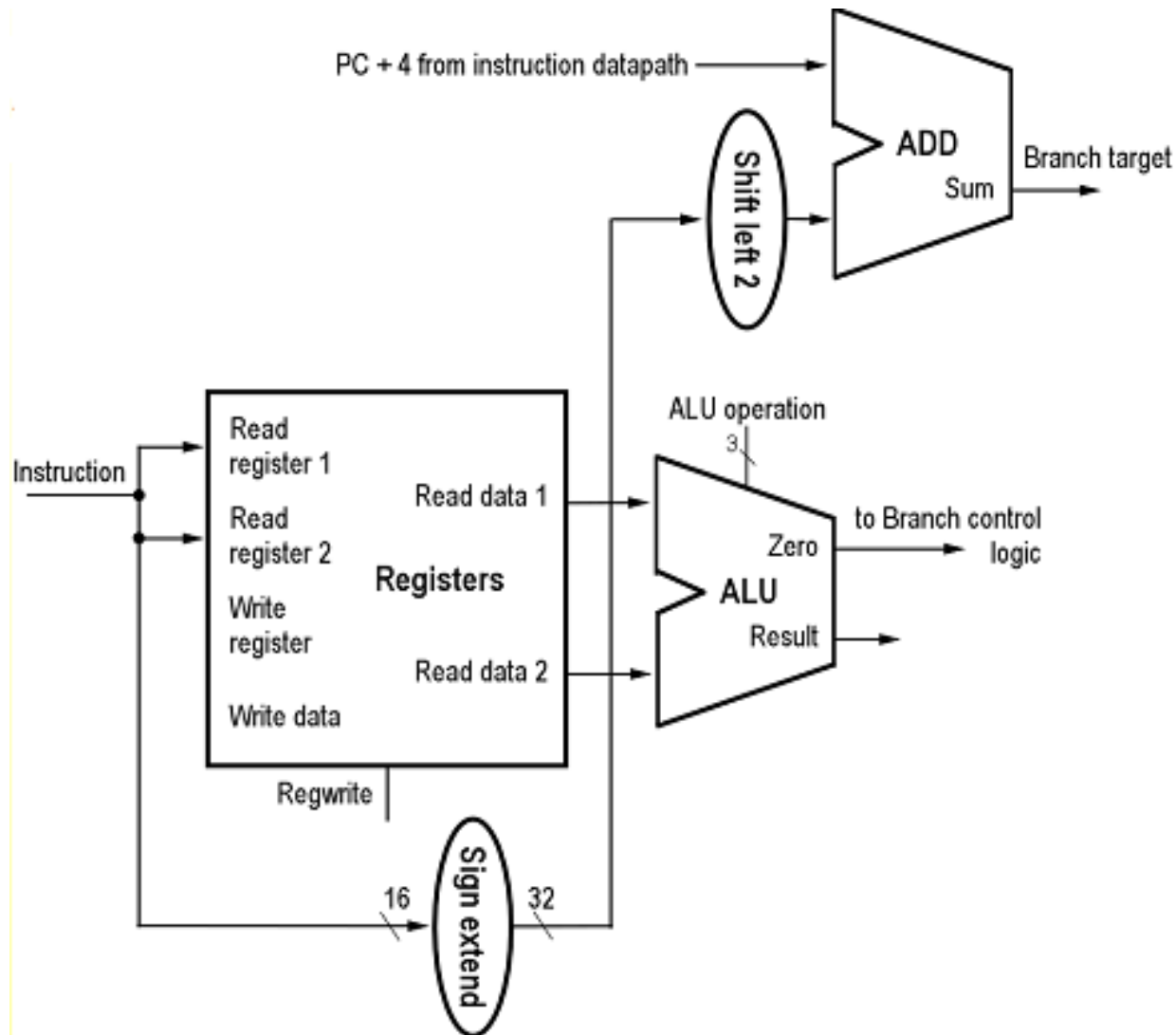
Datapath cho lệnh bộ nhớ (lw,sw)

62



Datapath cho lệnh rẽ nhánh (beq,j)

63



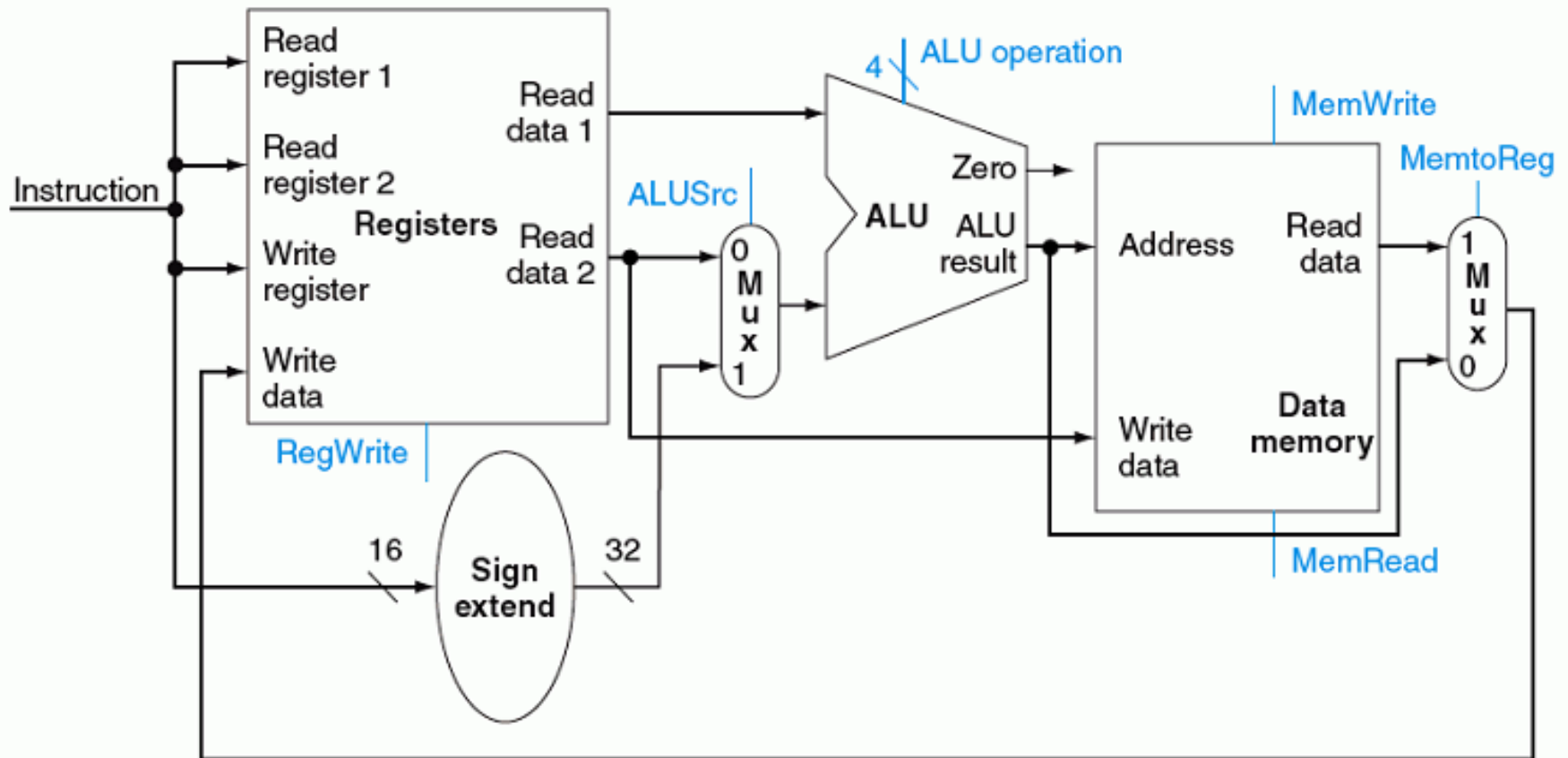
Datapath cho R-format ?

64

- Làm sao xây dựng Datapath cho R-format “xài chung” Datapath của I và J-format?
- Cần những bộ MUX đóng vai trò data selector để chia sẻ và lựa chọn những phần tử kiến trúc giữa những nhóm lệnh khác nhau
- Lưu ý: Hiện tại chúng ta chỉ xét CPU theo kiến trúc đơn chu kỳ (single cycle) – Mọi câu lệnh chỉ thực thi trong 1 chu kỳ clock

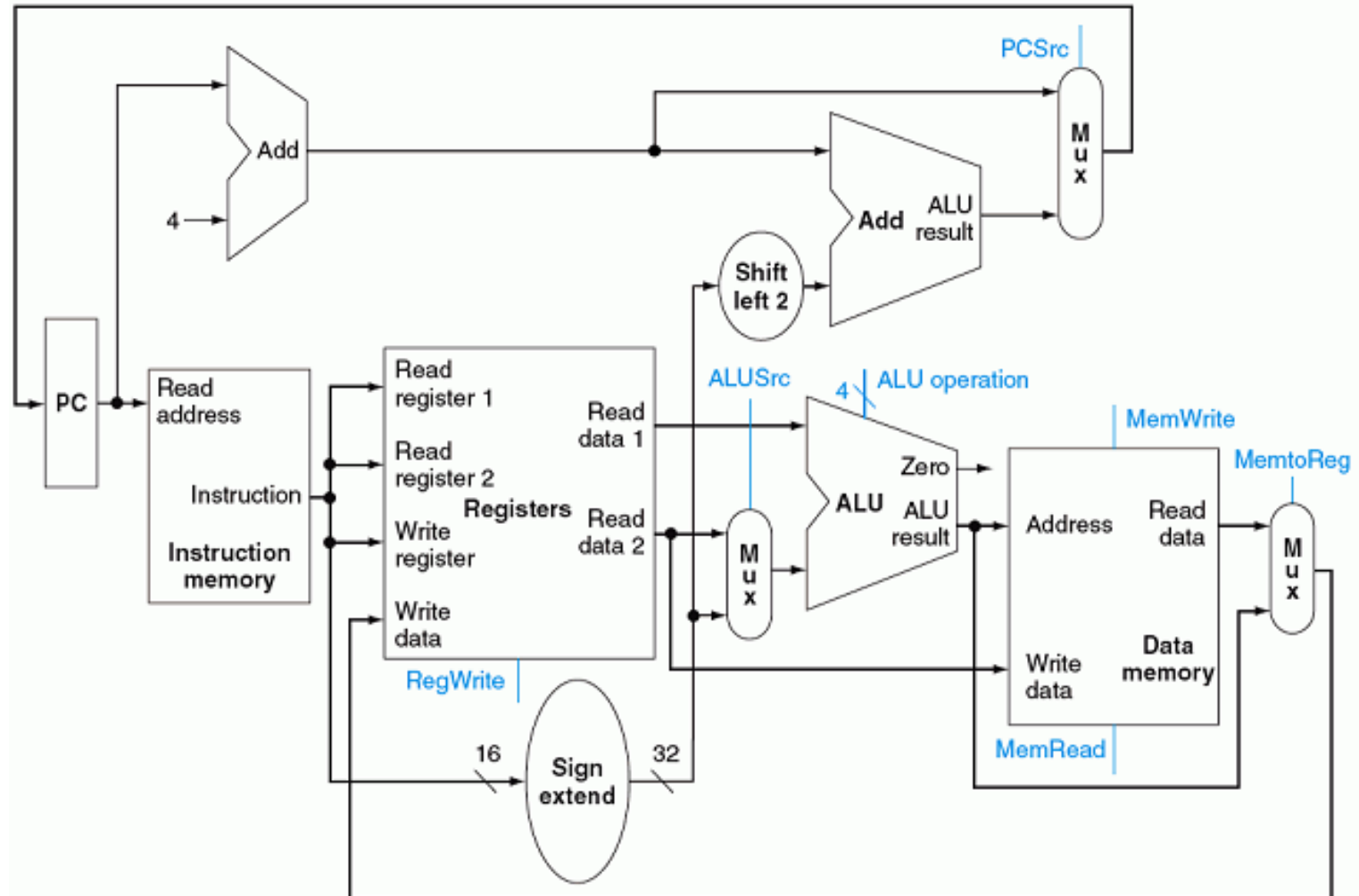
Datapath cho R-format

65



Datapath cho I,J,R-format

66



Tín hiệu điều khiển

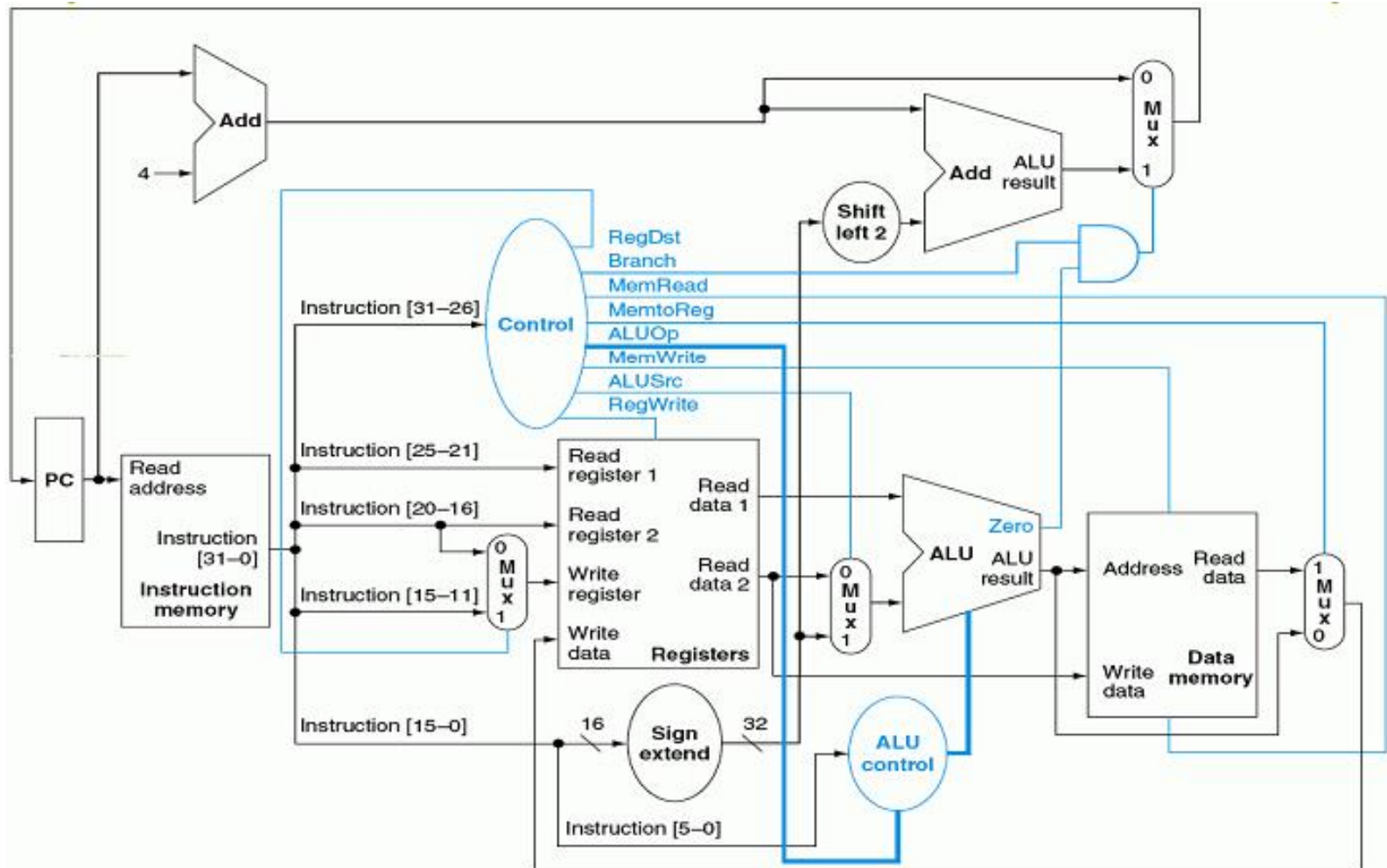
67

Signal name	Effect when deasserted	Effect when asserted
RegDst	The register destination number for the Write register comes from the rt field (bits 20:16).	The register destination number for the Write register comes from the rd field (bits 15:11).
RegWrite	None.	The register on the Write register input is written with the value on the Write data input.
ALUSrc	The second ALU operand comes from the second register file output (Read data 2).	The second ALU operand is the sign-extended, lower 16 bits of the instruction.
PCSrc	The PC is replaced by the output of the adder that computes the value of PC + 4.	The PC is replaced by the output of the adder that computes the branch target.
MemRead	None.	Data memory contents designated by the address input are put on the Read data output.
MemWrite	None.	Data memory contents designated by the address input are replaced by the value on the Write data input.
MemtoReg	The value fed to the register Write data input comes from the ALU.	The value fed to the register Write data input comes from the data memory.

Control unit ?

68

- ALU cần tín hiệu điều khiển hoạt động từ ALU Control



ALU Control Unit

69

- Các tín hiệu điều khiển ALU (4 bit):

ALU control Input	Function
0000	and
0001	or
0010	add
0110	sub
0111	slt
1100	nor

ALU Control Unit

70

Instruction (Control Unit → ALU Control)			ALU control input (to ALU)
Operation	ALU Opcode	Function	
lw	00	xx xx xx	0010 (add)
sw	00	xx xx xx	0010 (add)
beq	01	xx xx xx	0110 (subtract)
add (R-type)	10	10 00 00	0010 (add)
subtract (R-type)	10	10 00 10	0110 (subtract)
and (R-type)	10	10 01 00	0000 (and)
or (R-type)	10	10 01 01	0001 (or)
slt (R-type)	10	10 10 10	0111 (slt)

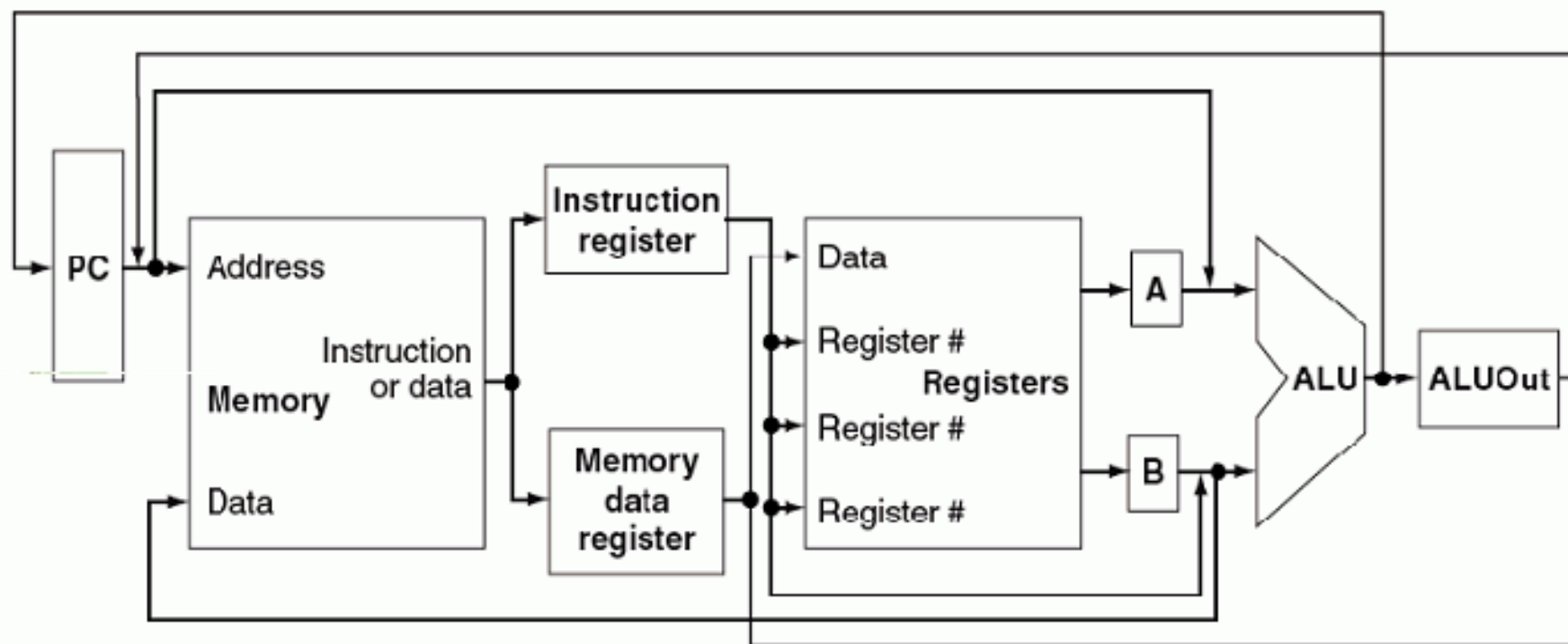
Bảng chân trị 4-bit ALU Control

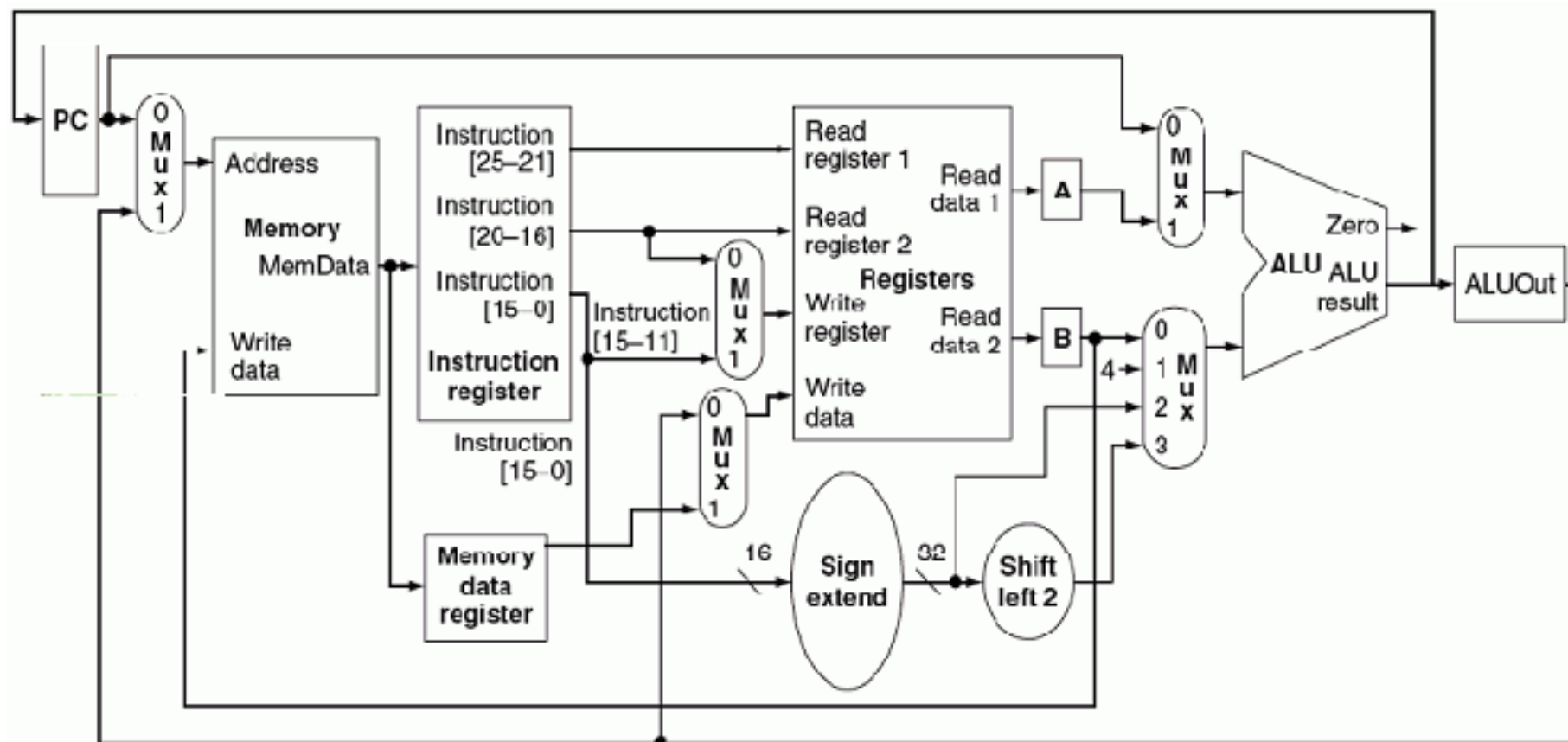
71

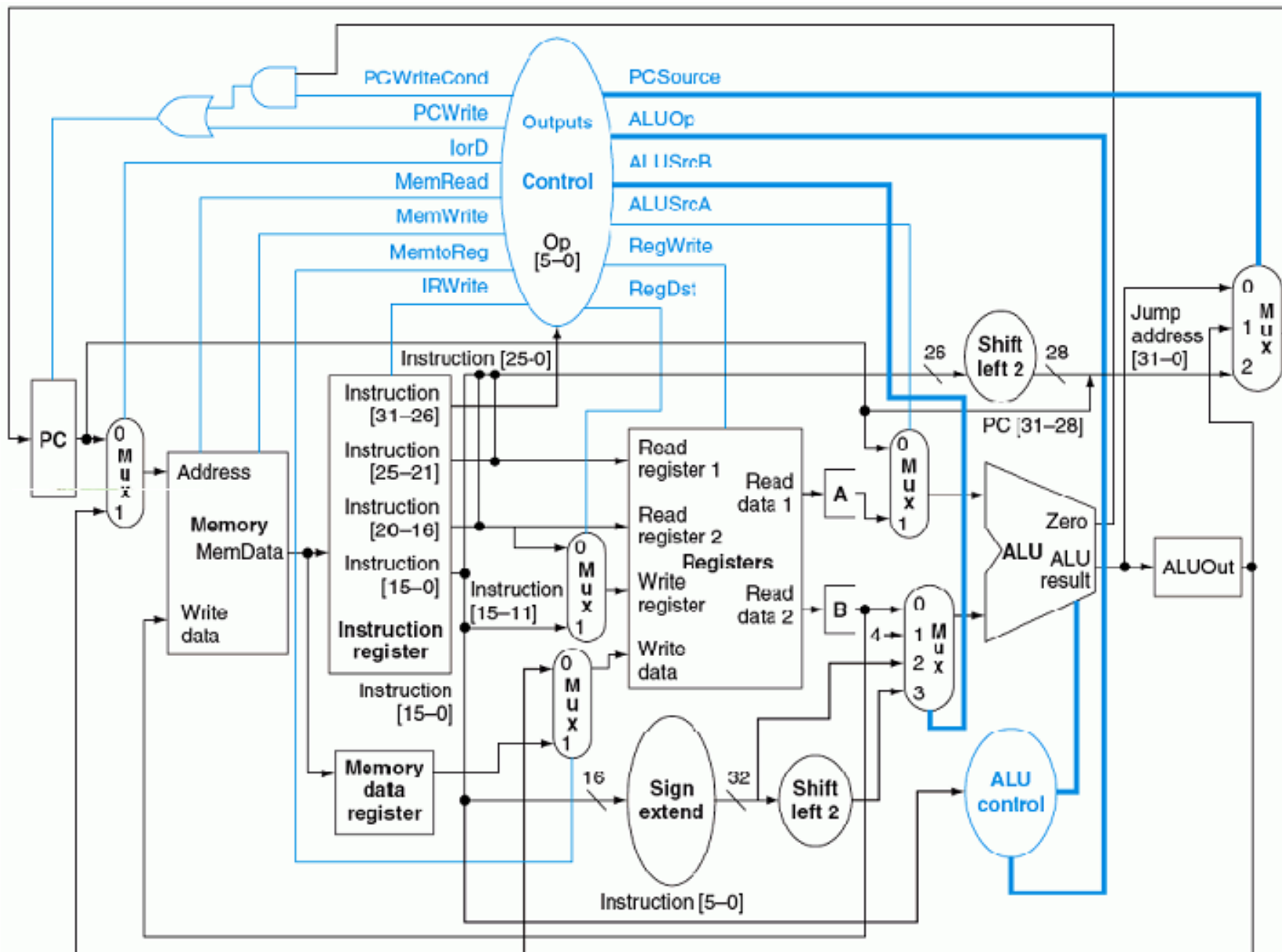
ALUOp		Funct field						Operation
ALUOp1	ALUOp0	F5	F4	F3	F2	F1	F0	
0	0	X	X	X	X	X	X	0010
X	1	X	X	X	X	X	X	0110
1	X	X	X	0	0	0	0	0010
1	X	X	X	0	0	1	0	0110
1	X	X	X	0	1	0	0	0000
1	X	X	X	0	1	0	1	0001
1	X	X	X	1	0	1	0	0111

CPU đa chu kỳ (multiple-cycle)

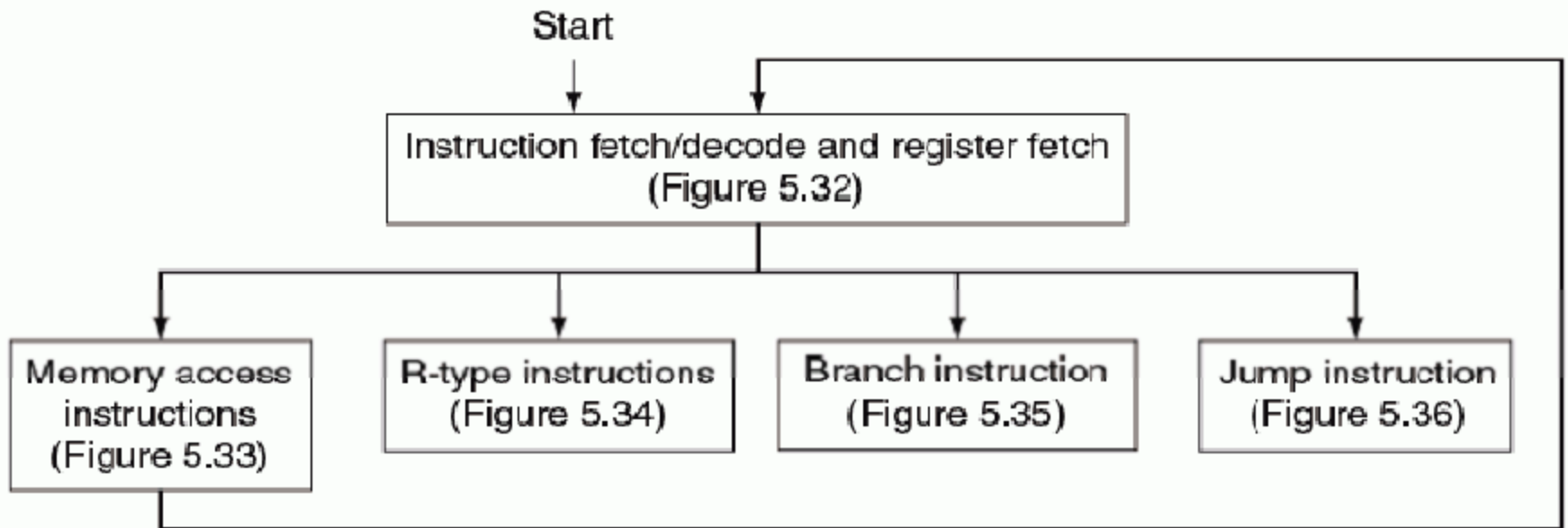
- Trong thực tế không sử dụng CPU single-cycle vì các lý do:
 - Thời gian thực hiện các câu lệnh luôn khác nhau → Phải chọn chu kỳ hoạt động của CPU bằng với chu kỳ thực thi câu lệnh dài nhất !
 - Khả năng trùng lặp các phần tử chức năng cao
- Ở CPU đa chu kỳ (multiple-cycle), quá trình thực thi 1 câu lệnh diễn ra thành nhiều chu kỳ clock
- Một số khác biệt so với single-cycle:
 - Tinh chỉnh thời gian thực thi từng câu lệnh theo gián đồ trạng thái
 - Có thể sử dụng 1 bộ nhớ chung cho cả câu lệnh lẫn dữ liệu
 - Thêm vào 1 số thanh ghi để chứa dữ liệu/kết quả trung gian

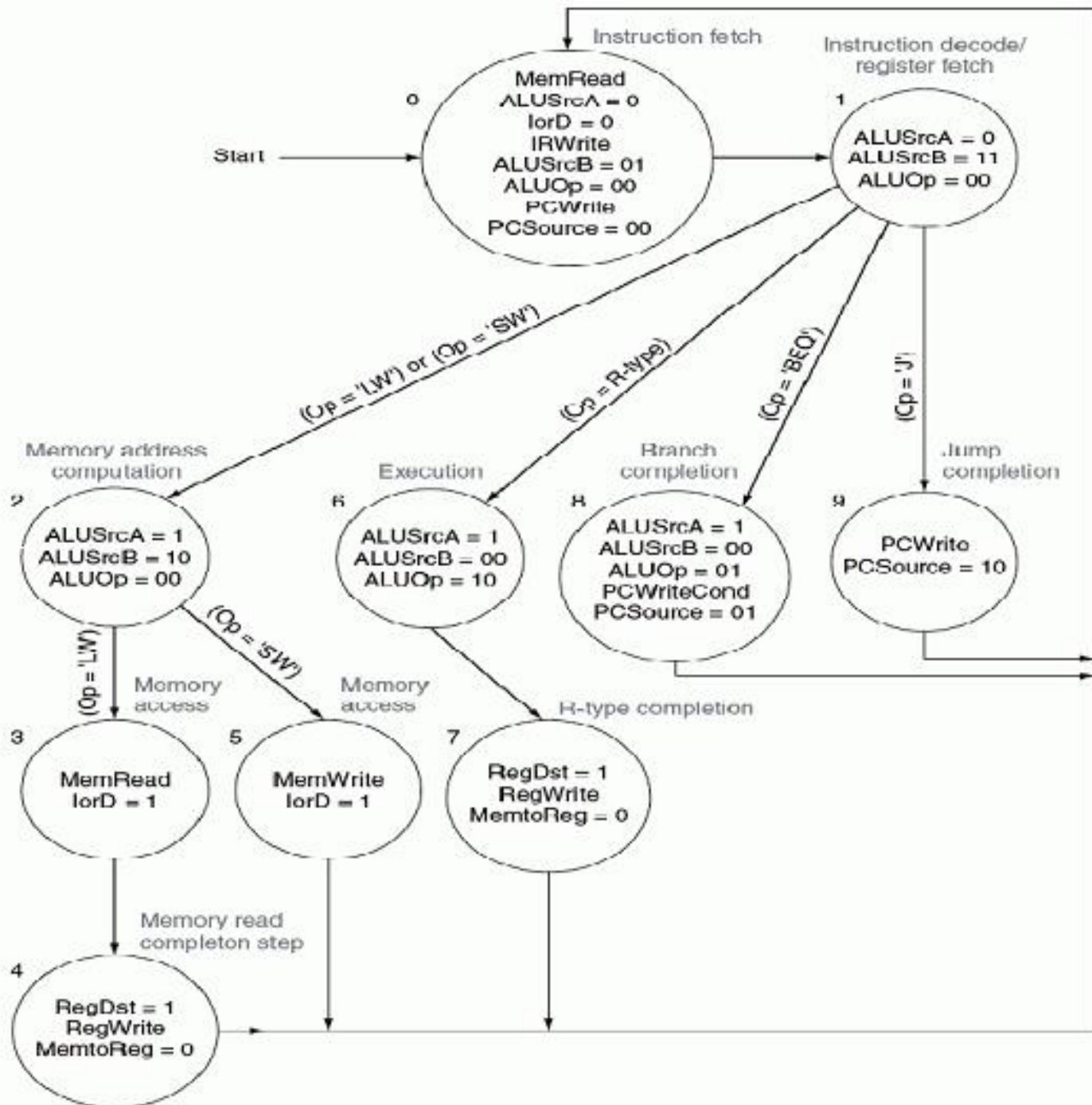






Quy trình thực thi lệnh





Homework

78

- Sách Pettersen & Hennessy: Đọc chương 5